



Master Dissertation

An Empirical Analysis of Build Time Optimization Strategies in GitHub Actions: Evaluating the Impact of Caching and Parallelization on CI/CD Pipeline Efficiency

Submitted June 2026, in partial fulfilment of the conditions of the award of the degree

Msc Software Engineering

By Sagar Gurung

School of Computing and Engineering

University of West London

I hereby declare that this dissertation is all my own work, except as indicated in the text:

Signature _____

Date: 01-06-2026

Copyright

The copyright of this dissertation belongs to the author under the terms of the UK Copyright acts as amended by The University of West London regulations.

Due acknowledgement must always be made of the use of any materials contained in, or derived from this thesis.

Abstract

In today's software development landscape, Continuous Integration and Continuous Deployment (CI/CD) pipelines have emerged as a cornerstone of the agile approach to software development, allowing a team to automatically build, test, and deploy software more quickly and reliably. But this is still a major challenge that slows down software releases, increases operational costs and decreases developer productivity through long build time. In this research, dependency caching and job parallelization are two of the most common build time optimization strategies in GitHub Actions workflows that are analysed.

The study examines 5-10 selected open-source repositories on the GitHub platform, which are chosen carefully to be the ones that utilize GitHub Actions as their CI/CD platform. This research is based on a systematic methodology with CI mining techniques and using thresholds, it identifies the common inefficiencies found in CI workflows like missing cache settings, running jobs sequentially when parallel execution is possible, and downloading the same dependencies multiple times. The identified inefficiencies are then corrected by specific optimization strategy and the improvement in the build time is quantified by performing statistical validation before and after the implementation.

It includes six methodological steps: a repository selection based on specific rules, a data collection by means of the GitHub API, an identification of the inefficient repositories by applying threshold-based techniques, an implementation of the optimization via repository forking, a measurement and a statistical analysis of the data based on Python libraries and a practical guideline development. Every workflow run configuration and build logs are gathered and analysed to provide baseline metrics prior to optimisations.

The expected outcomes are: quantitative measures of the time savings from caching and parallelization, categorization of common inefficiencies in real-world GitHub Actions workflows, and recommendations that developers can take action on for optimizing their CI/CD workflows. The results are added to a growing collection of empirical studies on CI/CD optimization, and are applicable to both open-source and commercial software development projects.

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Cain Kazimoglu, for his invaluable guidance, continuous support, and constructive feedback throughout this research project. His expertise and encouragement have been instrumental in shaping this dissertation.

I am also grateful to the module leader, Julie Wall, and all the teaching staff at the School of Computing and Engineering, University of West London, for providing an excellent learning environment and the knowledge foundation that made this research possible.

My heartfelt appreciation goes to my family and friends for their unwavering support, patience, and encouragement during my studies. Their belief in my abilities has been a constant source of motivation.

Finally, I would like to thank the open-source community and the developers who maintain the repositories analyzed in this study. Their commitment to open-source software has made this research possible and contributes to the advancement of software engineering practices worldwide.

Table of Contents

Copyright.....	2
Abstract.....	3
Acknowledgements.....	4
Table of Contents.....	5
Table of Figure.....	7
1. Introduction.....	1
1.1 Research Background.....	1
1.2 Research Problem.....	1
1.3 Research Rationale.....	2
1.4 Research Significance.....	3
1.5 Research Aim.....	4
1.6 Research Objectives.....	4
1.7 Research Questions.....	4
2. Literature Review.....	5
2.1 Foundations of Continuous Integration and Continuous Deployment.....	5
2.2 Research on GitHub Actions.....	6
2.3 Build Time Optimization Research.....	6
2.4 Research on Parallelization in CI/CD.....	8
2.5 Research Gap and Motivation.....	8
2.6 Summary.....	9
3. Methodology.....	11
Stage 1: Repository Selection.....	12
Stage 2: Data Collection and Baseline Analysis.....	14
Stage 3: Inefficiency Identification.....	16
4. Findings And Result.....	18
Stage 4: Optimization Implementation.....	18
4.1 Dependency Caching Implementation.....	18
4.2 Job Parallelization Implementation.....	20
4.3 Forking and Execution Methodology.....	22
Stage 5: Results and Analysis.....	23
5.1 Baseline Performance.....	23
5.2 stripe/stripe-python Results.....	24
5.3 django/djangoproject.com Results.....	26
5.4 nashtech-garage/yas Results.....	27
5.5 sparrowwallet/sparrow Results.....	29
5.6 Cross-Repository Comparison.....	30
5.7 Answering the Research Questions.....	33
6. Limitations.....	35
7. Conclusion.....	37

8. Future Work.....	41
9. References.....	43
10. Appendix.....	45
Appendix A: Github Links.....	45
Appendix B: Repository Selection Script.....	45
Appendix C: Workflow Verification Script.....	48
Appendix D: Baseline Data Collection Script.....	51
Appendix E: Post-Optimization Data Collection Scripts.....	54
D.1 stripe/stripe-python.....	54
D.2 django/djangoproject.com.....	55
D.3 nashtech-garage/yas.....	56
D.4 sparrowwallet/sparrow.....	57
Appendix E: Optimized Workflow Configurations.....	58
E.1 stripe/stripe-python — Caching.....	58
E.2 stripe/stripe-python — Parallelization.....	61
E.3 stripe/stripe-python — Combined.....	63
E.4 django/djangoproject.com — Caching.....	66
E.5 nashtech-garage/yas — Caching (cart service).....	68
E.6 nashtech-garage/yas — Parallelization (cart service).....	70
E.7 nashtech-garage/yas — Combined (cart service).....	72
E.8 sparrowwallet/sparrow — Caching.....	75

Table of Figure

Figure 1: Baseline Build Times Across Selected Repositories	23
Figure 2: stipe/stripe-python- Optimization Impact	25
Figure 3: nashatech-garage/yas - Optimization Impact	28
Figure 4: Effect of Dependency Caching Across All Repositories	30
Figure 5: Summary of Build Time Reduction Across All Repositories	31

1. Introduction

1.1 Research Background

The landscape of modern software development has been, kinda reshaped, by the adoption of Continuous Integration and Continuous Deployment (CI/CD) pipelines. These automated systems let development teams build, test, and deploy code changes in a quick and repeatable manner, and that becomes a key part of contemporary software engineering practice. As Shahin, Babar and Zhu (2017) note, CI/CD practices can enhance software quality while also shrinking the gap between writing code and giving it to end users. And with software systems getting more tangled everyday, plus the growing need for shorter release cycles, CI/CD stopped being only a “nice to have” best practice and turned into an operational necessity for most development organisations.

Now among the platforms that help run CI/CD, GitHub Actions has surfaced as one of the most popular since it launched in 2019. It’s deeply tied to GitHub, which already hosts millions of software repositories worldwide, and it lets developers craft automated workflows using YAML configuration files that run when certain events happen like code commits or pull requests. People tend to adopt it fast because it is free for public repositories, the configuration is fairly simple, and it also leans on a strong ecosystem of community-developed actions you can reuse (Decan et al., 2022). More recently, Rostami Mazrae et al. (2023) suggests GitHub Actions has actually overtaken older tools such as Travis CI and CircleCI in adoption, and it has basically become the default CI/CD option for a lot of open-source projects.

1.2 Research Problem

Even with the obvious advantages of CI/CD automation, long build times are still kind of a persistent problem, and it’s also widely reported. When pipelines or workflows take too long to wrap up, developers end up waiting around for feedback on their code changes, which messes with their train of thought, and it reduces overall productivity. Gallaba et al. (2022) reported that slow CI feedback often pushes developers into context switching, to unrelated tasks while they’re waiting on results. That behaviour is especially costly for

knowledge workers, at least in practical terms. And then there's the organisational side: prolonged builds can turn into higher computing costs, mainly for companies that depend on cloud-based CI services, because execution time is billed as a direct financial spend.

The scope here is pretty large. Bouzenia and Pradel (2024) looked at resource usage in GitHub Actions workflows and observed that one widely used open-source project can rack up CI costs of about \$504 each month. They also found that, with optimisations that are relatively straightforward, execution time can drop somewhere between 1.1% and 31.6% for the workflows that are impacted. This points to the idea that a lot of projects still have a lot of unrealised tuning potential inside their CI/CD setup. Still, despite built-in options like the actions/cache action for dependency caching, and the native support for parallel job execution, many real-world workflows don't really use these features fully (Gallaba and McIntosh, 2020).

1.3 Research Rationale

Two of the techniques that are very often suggested to help lower CI/CD build times are dependency caching and job parallelisation. Dependency caching basically saves the outcome of operations that take a while, like pulling external libraries from the network, so you don't have to redo that same work every time if the dependencies have not changed. Job parallelisation meanwhile lets the separate bits of a workflow run at the same time, instead of doing everything one after another, and because of that the total elapsed time tends to shrink. GitHub Actions already has built in support for both of these, but why people don't use them as much as they could is ... different depending on who you ask. Some developers may simply not know those optimisation features exist, while others might have the skills but still can't quite implement them the right way, or they're not sure how to wire them up. Also, there is a shortage of empirical evidence that actually measures the real effect of caching and parallelisation in real projects, so it becomes hard to say "yes" to the engineering effort to create and keep tuned configurations.

So this mismatch, between having optimisation options available and actually adopting them, is kind of the main reason for this research. Without solid, evidence-based guidance, developers end up using mostly stories, tips, or documentation snippets that might not match the many kinds of real project setups. An empirical study that checks the effect of caching plus parallelisation across multiple repositories and programming languages could offer the kind of measured, quantified signal that seems to be missing in the current literature.

1.4 Research Significance

This work kind of contributes to software engineering, both in academia, and in the field use. In a more academic sense, it grows the existing CI/CD literature a bit, not just by describing things, but by adding empirical data about how well certain optimisation techniques actually work on real GitHub Actions workflows. Earlier papers , like Gallaba and McIntosh (2020) and Bouzenia and Pradel (2024), focused more on CI workflow features and resource consumption. But, as far as is clear, fewer studies have really put optimisations into practice in a systematic way and then measured the improvement using a controlled before-and-after comparison. In other words, this research tries to close that gap by using a fairly strict pre-test/post-test approach across a varied set of open-source repositories.

From the practical angle, the results are meant to help software development teams who want to push their CI/CD pipeline performance forward. The study outputs evidence-based guidelines that say, in plain terms, when each optimization technique tends to help most, how to apply it correctly, and roughly how big the gains might be. These recommendations are not just theoretical advice, they are based on actual measurement , so they're meant to be used, not merely discussed. Also, there are cost implications that are not small: if you reduce build time, you typically also reduce CI/CD infrastructure spending, you get faster developer feedback loops, and the overall team productivity usually improves, even if indirectly.

1.5 Research Aim

This research is about doing an empirical analysis of build time optimisation approaches in GitHub Actions workflows, mostly to see the measurable effects of dependency caching and job parallelisation, in a clear way. We will do it by collecting data in a systematic manner, then implementing the optimisations in a controlled manner, and finally using statistical analysis. The goal is to put numbers on how effective these strategies are across a wide mix of open source repositories, and also to come up with workable, practical guidance for developers who want to speed up their CI/CD pipeline performance.

1.6 Research Objectives

To achieve this aim, the following objectives have been defined:

1. To identify and select suitable open-source repositories that use GitHub Actions, covering multiple programming languages and representing a range of project types and CI/CD configurations.
2. To collect and analyse workflow configurations and build execution data from the selected repositories in order to establish baseline performance metrics.
3. To identify and categorise common inefficiencies in the existing GitHub Actions workflows, focusing on caching and parallelisation deficiencies.
4. To implement dependency caching and job parallelisation optimisation strategies on forked copies of the selected repositories and measure their individual and combined impact on build times.
5. To perform statistical analysis to determine whether the observed improvements are statistically significant.
6. To develop practical, evidence-based guidelines and recommendations for developers seeking to optimise their GitHub Actions workflows.

1.7 Research Questions

The following research questions guide this investigation:

RQ1: To what extent does the introduction of dependency caching and job parallelisation reduce build times in GitHub Actions workflows, individually and in combination?

RQ2: What practical guidelines can be derived from the empirical findings to assist developers in optimising their CI/CD pipelines?

2. Literature Review

The importance of efficient CI/CD pipelines has been recognized in academic research for quite a while, you know, and there is a growing stream of empirical studies that look at continuous integration practices, build optimization, and how developers actually experience CI/CD tools. In this literature review, we sort through the existing research that is relevant to build time optimization specifically for GitHub Actions, then we point out where the current knowledge is a bit thin , and we set up the theoretical grounding that this study builds on.

2.1 Foundations of Continuous Integration and Continuous Deployment

Continuous Integration sort of emerged as a software development practice in the late 1990s, and since then it became this kind of cornerstone of modern software engineering. Shahin , Babar and Zhu (2017) did a comprehensive systematic literature review. In it they looked at approaches, tools , challenges and day to day practices in CI/CD. In general their study found that CI/CD adoption tends to bring faster time-to-market, better software quality, and in a way smoother team collaboration. Still, they also pointed out that some issues keep showing up. Like the pipeline setup being complicated ,and the fact that you have to keep maintaining things as a project kind of evolves.

On top of that, CI/CD platforms have evolved too, it looks like there was a real shift from self-hosted solutions such as Jenkins to more cloud based services. Rostami Mazrae et al. (2023) used qualitative research via interviews with experienced developers , aiming to understand CI/CD tool usage patterns and the choices behind migration. What they saw was that developers often pick CI/CD tools based on how well they integrate with systems they already use rather than purely on performance features. So, once a team

adopts something like GitHub Actions, they may not really dig into all the optimization capabilities that are there , at least not fully, not right away.

2.2 Research on GitHub Actions

Since GitHub Actions was introduced , it has gotten a lot of research attention, like really a lot. Decan et al. (2022) did a mixed method style analysis looking at how GitHub Actions workflows change across time. They examined 31,000+ repositories with more than 131,000 workflow change histories, and well the numbers are pretty big. What they reported was that most repositories tend to hold a median of two workflow files , and 8.4% of the workflow files are edited every week. That suggests continuous maintenance and some steady evolution in CI/CD setups, even if the change cadence is not always dramatic.

Then, Ghaleb, Hassan and Zou (2024) shifted focus toward complexity, heterogeneity, and compliance. In their study they found that a lot of workflows are more tangled than they need to be , and they often skip recommended best practices. They also argued that developers need better tooling and clearer documentation, because right now it's not always obvious how to do things in a more streamlined way. This result lines up pretty well with the motivation behind this work, since we are trying to provide practical guidelines for optimization.

Finally , Faqih et al. (2024) analyzed how CI/CD tools are actually used inside GitHub Actions workflows. They grouped the tools into several categories, like version control, static analysis, build automation, and test automation. Their outcomes showed that build automation tools show up most often , while test automation is adopted less frequently, for whatever reasons. Overall, this helps frame what developers commonly configure today and where inefficiencies can creep in, even when everything still “runs” in the end.

2.3 Build Time Optimization Research

Research on CI build optimization kinda has a long history. Gallaba and McIntosh (2020) did an influential study looking at how people use, and sometimes misuse CI features in Travis CI, they analyzed thousands of projects. From their results, it looked like many

developers never really take full advantage of the available optimization features, and misconfigurations show up a lot, which in turn causes these unnecessary build delays. Their work also set up important methods for mining CI configuration data, and that basically becomes the base for the way this current research is framed.

Then, building on that earlier foundation, Gallaba et al. (2022) suggested a way to speed up CI builds by caching the build environment, and also inferring dependencies. They created a tool named Kotinos, which automatically spots chances for caching, as well as ways to avoid running steps that are unchanged. Their evaluation reported that at least 87.9% of the builds in the studied set had some form of acceleration opportunity, so not just a tiny portion. Overall, this line of research demonstrated that big improvements can happen when the optimization strategy is chosen correctly, so it gives solid motivation for why the current study is worth doing.

More recently, Bouzenia and Pradel (2024) focused specifically on resource usage inside GitHub Actions workflows. They also claimed, and seemed to support it with evidence, that their work is the first comprehensive empirical study on resource usage and optimization opportunities for this platform. Their findings indicated that CI/CD brings noticeable costs, and that even small adjustments might yield meaningful time savings. One specific idea they evaluated was turning off scheduled workflows when repositories are inactive, which could reduce execution time by 1.1% to 31.6% for the workflows that are affected. Still, their emphasis was largely on describing resource usage patterns rather than measuring the direct impact of specific optimization techniques, like caching or parallelization, in a more targeted way.

Research on Caching in CI/CD

Caching has been picked out as one of the most useful techniques for cutting CI build times. The main idea is kind of simple: if you keep the outputs of costly steps (like downloading dependencies), then the next runs can reuse those cached artifacts, instead of doing the whole thing again. GitHub Actions also has built in caching support via the actions/cache action, and that lets workflows stash and bring back files using cache keys that you set up.

Gallaba, Lamothe and McIntosh (2022) looked at eight years worth of operational records from CircleCI, which is another well known CI service. In their exploratory case study, they basically examined how caching, along with a few other performance enhancements, are actually applied in day to day use. They concluded that caching can be quite helpful, yet how well it works really relies on the way it's configured, including the cache key selection part and the cache invalidation strategy, which needs to be thought through.

2.4 Research on Parallelization in CI/CD

Job parallelization is another big optimization lane for CI/CD pipelines. If your workflow has a bunch of jobs that basically don't rely on each other, launching them at the same time can really shrink the overall build duration. GitHub Actions allows this via the workflow syntax, where the jobs tend to run in parallel by default unless you clearly say otherwise by declaring dependencies with the "needs" keyword, so yeah it's all kinda implicit until you make it explicit.

Yin et al. (2024) built a detection method aimed at catching developer-applied speeding patterns in CI. They assembled a pattern catalog, covering how people try to accelerate their runs, and that includes different parallelization strategies too. What they found was that these acceleration moves actually do show up in real projects, but the way they're used varies a lot from one repo to another, so it looks like people need more straightforward direction and best practices rather than just assuming everyone will do it the same way.

2.5 Research Gap and Motivation

Even though earlier research has already shown how important CI/CD optimization is, and it has also pointed out several techniques, there is still a notable hole in empirical studies. In particular, very few papers measure what caching and parallelization really do, inside GitHub Actions workflows. A lot of the existing work tends to look at other systems such as Travis CI or CircleCI, or they talk about GitHub Actions in a more general way, without actually checking the specific gains from optimizations.

On top of that, there's also a shortage of hands-on, proof based guidance for developers who want to improve their GitHub Actions workflows. Sure, GitHub has documentation about the available features, and you can find industry blogs that suggest many ideas. But, there is limited empirical evidence that numbers out the actual improvements you get from each concrete strategy. This study is meant to close that gap, by running a systematic empirical analysis and then delivering quantified measurements of how much build time can be reduced.

The findings of this literature review demonstrate that:

1. CI/CD optimization is an important research area with practical implications;
2. caching and parallelization are recognized as effective optimization techniques;
3. Many real-world workflows do not fully utilize available optimization features; and
4. There is a need for empirical research specifically focused on GitHub Actions optimization.

This study addresses these needs by providing quantified evidence of optimization effectiveness and developing practical guidelines for implementation.

2.6 Summary

This chapter's literature reviewed points at a few key themes that feel relevant to the research that comes next. CI/CD practices have kinda become part of modern software development, and in particular GitHub Actions has turned into the main platform people use to automate build, test and deployment workflows (Decan et al., 2022; Rostami Mazrae et al., 2023). A lot of the research agrees that long build times really bring meaningful costs for development teams, not only because of lost productivity from context-switching (Gallaba et al., 2022), but also due to direct money spent on cloud CI infrastructure (Bouzenia and Pradel, 2024). Dependency caching, and also job parallelisation, are often seen as good ways to shorten build durations, and GitHub Actions kinda supports both of these approaches fairly well, via the actions/cache action and through parallel job execution.

Still, Gallaba and McIntosh (2020), and also Yin et al. (2024), suggest that a large chunk of real-world workflows simply do not use these techniques. On top of that, the literature doesn't provide much hard empirical evidence that actually measures what happens when they are applied to different kinds of, real projects. So there's this gap between what is theoretically available in terms of optimisation features, and what has been proven in practice. That mismatch is what motivates the current study, basically, to add the quantitative evidence that is still missing in the literature today.

3. Methodology

This research uses a numbers based, empirical style that is sort of organized around a pre- test and post- test thing. Overall, the work is laid out in six back to back stages, starting with repository selection, then baseline data capture, then inefficiency spotting, optimization rollout, post-optimization measurements are taken, and finally practical guidance is built. For every repository that gets looked at, the baseline build performance is noted before any changes happen. After that, the optimization moves—namely dependency caching along with job parallelization—are applied to forked versions, and then the same metrics are measured again after everything is modified. The gap between the pre-optimization results and the post-optimization results is what gives the quantitative reason for judging how well each technique actually works.

This empirical design was picked for a few reasons, and well, it sounds simple but it matters. First, the build time optimization bit is inherently measurable, and using a quantitative approach helps you do an objective comparison of performance across different repositories and strategies. Second, since we work with real world open-source projects instead of synthetic benchmarks, the results should have more external validity, and they'll feel more useful in practice. Third, the pre test / post test layout means each repository ends up acting as its own control, which can lower the impact of nuisance factors like codebase size differences, language variance or infrastructure shifts.

Overall, the approach matches what was done in earlier CI/CD research, including Gallaba and McIntosh (2020) about CI workflow analysis and Bouzenia and Pradel (2024) regarding GitHub Actions resource consumption. For the statistics, they use paired t-tests, and when normality isn't satisfied then they switch to Wilcoxon signed-rank tests, with $\alpha = 0.05$ as the cutoff. Everything—scripting, data gathering, and the statistical work—is done in Python, with PyGithub, Pandas, SciPy, and Matplotlib.

Stage 1: Repository Selection

Repository selection happened using a custom Python script, basically glued together with the PyGithub library. It went and called the GitHub Search API, for open source repos that matched a bunch of conditions at once , GitHub Actions used as the main CI/CD setup, then a star number somewhere between 100 and 2,000 . The goal was to focus on mid-sized codebases, the kind that feel close to what a typical team would run into day to day . There was also a cutoff of at least 50 workflow runs recorded, so there's enough history for later stats, plus proof of active maintenance, meaning commits showed up in the last six months or so. Lastly, the candidates had to represent three programming languages, Python plus JavaScript and also Java . The 2,000 stars cap was not accidental. We set it on purpose to filter out those big enterprise level behemoths, where CI/CD usually turns into a whole ecosystem: many interconnected workflow files, extra custom runner infrastructure, and often proprietary tooling too, which then limits how generalizable the conclusions can really be for ordinary development teams. Beyond that main filter, the script also did a kind of quick workflow configuration scan, checking if caching like the actions/cache action was already present, or if the repo had matrix-style build strategies wired in. It also removed repositories where the average build time landed under 10 seconds, because those super short pipelines wouldn't really give much room for optimization insights, or meaningful comparisons in the first place.

The search gave 30 candidate repositories across the three languages, sort of. From this big pool, five were picked in order to end up with a balanced dataset on three dimensions language diversity, like making sure there was at least one repository for each target language. Also optimization opportunity, this time trying to mix repositories that already had caching and parallelization with ones that didn't. And then build duration, where the emphasis was on repositories with build times that were long enough, so improvements could be actually measured. After the first selection, there was a verification step, kind of like a double check. In that step, a second Python script downloaded and parsed the YAML workflow files from each repository. That confirmation matched what was found earlier during the search, like the number of jobs

per workflow, whether dependency caching was present or absent, the use of matrix strategies, and how job dependencies were arranged.

Table 1 shows the five selected repositories and their main traits, so you can compare them quickly.

Table 1: Selected Repositories and Their Characteristics

Repository	Language	Stars	Runs	Avg Build (s)	Caching	Matrix
stripe/stripe-python	Python	1,987	2,444	198.0	No	Yes
django/djangoproject.com	Python	1,995	4,219	109.0	No	No
pubkey/broadcast-channel	JavaScript	1,992	722	306.4	Yes	Yes
nashtech-garage/yas	Java	1,955	2,496	258.0	No	No
sparrowwallet/sparrow	Java	1,971	59	510.0	No	Yes (OS)

The selected repositories span different project types. stripe/stripe-python is the official Python SDK for the Stripe payment processing platform, and it has this multi-job CI workflow with lint, build, test and publish stages. django/djangoproject.com is basically the source code for the official Django project website, with a single-job test workflow. pubkey/broadcast-channel is a JavaScript library for cross-tab and cross-process communication, its workflow already uses both dependency caching and matrix builds , so it was put in on purpose as a kind of control case, a reference point for judging the unoptimized repositories. nashtech-garage/yas is a Java based microservices e-commerce platform, and it includes 53 separate workflow files, one per service, none of those workflows do any caching or parallelization. sparrowwallet/sparrow is a desktop Bitcoin wallet application, it builds on three operating systems using a matrix strategy, but it still has no dependency caching. Taken together, those five repos give four intervention targets, and one control, so the research can see how optimization strategies behave across languages, architectures and project scale sizes.

Stage 2: Data Collection and Baseline Analysis

After the repositories were confirmed, the second phase sort of zeroed in on putting a quantitative baseline in place, so later post optimization results could actually be compared. A little data collection script was built, it was meant to talk to the GitHub Actions API and pull back run level details from recent workflow executions that had gone through successfully. For each repository, we aimed for at least 30 completed runs that were flagged as successful, because that number was considered enough for the next statistical checks. We only kept successful runs, on purpose, to not distort the baseline with builds that fail early due to code issues rather than reflecting real build behaviour. For nashtech-garage/yas, which has 53 distinct service workflows, the collection was done through five representative services—cart, product, order, customer, and inventory. That choice was intended to capture a broader spread of build behaviour across the microservice setup, while still staying within the practical constraints tied to GitHub API rate limits.

Each record basically captured the total workflow duration (in other words the elapsed time between the run's creation and its completion), plus the timing of each individual job inside that workflow, also the run number, the Git event that kicked off the run (like a push or pull_request), and the branch name. Everything got stored in CSV format so later analysis would be easier. The starting dataset had 167 workflow runs across five repositories. Then during data cleaning, eight records showed up as anomalous, with durations over one hour. When we dug into it, we found out those outliers were workflow runs that stayed in a queued state for a long while, instead of actually doing work, which is a well known trait of GitHub Actions shared runner infrastructure when there is high demand. So we removed those, and we ended up with 159 valid observations for the baseline analysis. Descriptive statistics for every repository were computed and they're laid out in Table 2.

Table 2: Baseline Build Time Statistics

Repository	Runs	Mean (s)	Median (s)	Std Dev (s)	Min (s)	Max (s)
stripe/stripe-python	29	262.9	264.0	35.5	199.0	345.0
django/djangoproject.com	30	132.9	133.0	6.8	121.0	150.0
pubkey/broadcast-channel	30	306.4	286.0	51.4	258.0	461.0
nashtech-garage/yas	40	353.1	251.0	286.4	184.0	1,298.0
sparrowwallet/sparrow	30	460.8	441.5	54.0	412.0	660.0

From the baseline data , you can see pretty clear performance vibes across the different repositories. django/ djangoproject.com seems to be the steadiest, the build results are tightly bunched , with a standard deviation of 6.8 seconds, plus the mean and median are almost the same, so the pipeline feels stable and almost boring in a good way, minimal run-to-run wobble. Then stripe/stripe-python is somewhat steadier? not as much, it has a standard deviation of 35.5 seconds, and its mean sits at 262.9 seconds, so it kind of lands in the middle band of the whole dataset. For sparrowwallet/sparrow , the picture shifts: the longest mean build time is 460.8 seconds. That's mostly because it does multi-platform matrix builds across Linux , macOS, and Windows , and that longer tail makes it a strong contender for showing how dependency caching can change things in practice. nashtech-garage/ yas is the one that jumps around the most, it shows the highest variability, with a standard deviation of 286.4 seconds. that spread lines up with its microservice workflows being different from each other: the inventory service averages 233.3 seconds per build, while the product service averages 518.2 seconds. So yeah, that gap is probably driven by how large each service's dependency graph is, and also by how hefty the test suite becomes. As a control case, pubkey/broadcast-channel already has both caching and matrix builds in place. It lands at a mean of 306.4 seconds, and a standard deviation of 51.4 seconds. Taken together , these baseline measurements basically act as the pre-intervention reference points

that we'll compare all the post-optimization results against later on, once the next stages of the research kick in.

Stage 3: Inefficiency Identification

During Stage 1, they verified the workflow, and then later on each repository's CI/CD setup was looked at a bit more carefully (kinda systematically, i guess) to find those particular inefficiencies that are really worth optimizing. The review was mainly focused on two categories of trouble , first the lack of dependency caching, and second the not-so-great use of parallelization, all based on the criteria that the research methodology already laid out. For caching, a workflow was tagged as inefficient if it ran dependency installation commands—like `pip install`, `npm ci`, or `mvn install`—while also not using any kind of caching mechanism along with it. In the parallelization part, a workflow was marked inefficient when jobs that should be independent were instead configured to run one after another, or when there was no matrix strategy in place even though one could be used in a pretty reasonable way.

Regarding cache stuff, it was observed that in four out of the five repositories, there is no real dependency caching to speak of. For example, `stripe/stripe-python` and `django/djangoproject.com` both end up running `pip install` on every single workflow run, and they do it without any caching for the `pip` download directory so the same packages get downloaded again and again across later builds. Then `nashtech-garage/yas` kinda takes it further, because it shows the strongest caching shortfall: all 53 of its service workflows run Maven build commands like `mvn install`, `mvn verify`, and `mvn package` but they don't cache the local Maven repository at `(~/.m2/repository)` . Because of that, the dependency tree, which can easily include hundreds of library artefacts, is re-resolved and re-fetched from remote repositories on each run. `sparrowwallet/sparrow` also misses caching for its build tool dependencies, and this gets worse since those dependencies have to be fetched separately for each operating system target in the matrix, so it repeats three times over. The only “control” repo, `pubkey/broadcast-channel`, already has caching set up through `actions/cache` for its `npm` dependencies, so that part is at least handled properly.

On the topic of parallelization, the analysis sort of pinpoints a few structural inefficiencies. For stripe/stripe-python, the setup in stripe/stripe-python defines a dependency chain where the publish step is told to wait for build, test and lint to finish, more or less in a strict line. That said, lint and test do not really have any shared data dependency, they could run at the same time, and this would shrink the total workflow length, since the two longest independent parts could overlap. In django/djangoproject.com, all testing is packed into one monolithic job, and there is no matrix approach, so the chance to split tests across different Python versions or operating systems in parallel is basically lost. Then nashtech-garage/yas follows a similar template but on a larger scale, each of its 53 microservice workflows has just a single build job, which handles compilation, testing and verification in sequence, all inside one runner, and again no matrix setup to check multiple Java versions. sparrowwallet/sparrow, meanwhile, already uses an operating system matrix to compile across Linux, macOS, and Windows at once, so its remaining parallelization needs are mostly already covered. Table 3 summarises the optimization strategies that can apply to each repository, and that table becomes the groundwork for the intervention stage that comes next.

Table 3: Optimisation Applicability Matrix

Repository	Caching	Parallelization	Combined	Role
stripe/stripe-python	Yes (pip)	Yes (job chain)	Yes	Test both strategies
django/djangoproject.com	Yes (pip)	Yes (single job)	Yes	Test both strategies
pubkey/broadcast-channel	Already present	Already present	N/A	Control
nashtech-garage/yas	Yes (Maven)	Yes (all services)	Yes	Largest scope
sparrowwallet/sparrow	Yes (build tools)	Partial (has matrix)	Partial	Caching primarily

4. Findings And Result

Stage 4: Optimization Implementation

After noticing a few inefficiencies in Stage 3, we took a pretty direct route— we implemented optimization strategies on forked copies of each repository. The whole idea of this phase was to apply the two techniques we were studying, dependency caching and job parallelization, but in a controlled way so we could see what happened both individually and together, for each repository. In practice, for every repo, the original workflow YAML files were adjusted to add one or both of the strategies, depending on what the inefficiency review actually uncovered.

Importantly, all changes were done only on those forked copies, so the original open source projects stayed 100% untouched, like completely unaffected for the duration of the work. When both approaches seemed useful at the same time, we didn't just mix them and hope for the best. Instead, three separate setups were prepared and then tested one after another, first the caching-only configuration, then the parallelization-only configuration, and finally a combined configuration where both strategies were active together at once.

4.1 Dependency Caching Implementation

Dependency caching was added with the `actions/cache@v4` action, which is a first party action by GitHub that is maintained for use inside GitHub Actions workflows. Basically this action saves the directories you name into a cache, and that cache sticks around across multiple workflow runs. The cache is tied to a key you define, which usually includes a hash from the project's dependency lock or spec files. So, on a later run if the key matches, those cached bits come back (restored) instead of downloading things again or rebuilding everything from scratch, and because of that the dependency resolution and installation phase gets noticeably faster.

This caching plan was adjusted for each project's package manager and language ecosystem. For the two Python repositories, `stripe/stripe-python` and `django/djangoproject.com`, the pip cache folder at `~/.cache/pip` was what got cached.

The cache key was put together using the runner operating system plus the hash of the project's dependency definition files like `setup.py`, `pyproject.toml`, or `requirements.txt`, which means the cache should get invalidated automatically whenever your dependency list changes. There were also restore keys set up in a sort of hierarchical way, so if the full key did not match, you could still pull a partial cache back as a kind of backup. That way even after small edits to a lock file the install step can still get a head start rather than starting cold. For `django/djangoproject.com` in particular, there was an extra cache entry for the Playwright browser binaries located in `~/.cache/ms-playwright`, since installing those browsers is a big part of the overall setup time.

For `nashtech-garage/yas`, the local Maven repository (`~/.m2/repository`) was cached using a key that was basically derived from the hash of the project's `pom.xml` files. Maven dependency resolution, which is widely known as one of the slowest pieces in Java CI pipelines, takes time because it has to download, maybe hundreds of library artefacts, from remote repositories, check their integrity, and then keep them locally. If there is no caching, that whole download cycle happens again on every workflow run even if nothing really changed in dependencies. The same caching setup was applied the same way across the cart, product and order service workflows, just so the results could be compared cleanly across the microservice architecture.

For `sparrowwallet/sparrow`, the Gradle cache directories (`~/.gradle/caches` and `~/.gradle/wrapper`) were cached using a key derived from the hash of the Gradle build files and wrapper properties. Gradle already keeps a local stash of downloaded dependencies and compiled build scripts, and when you restore that cache you avoid re-downloading dependencies plus you don't have to re-compile the build logic every single time.

In all these cases, the cache key strategy used a sort of layered structure: it combined the runner operating system, a workflow-specific identifier, and the hash of the relevant dependency lock files (or in the Maven case, the `pom.xml` set). That makes sure the cache gets invalidated when dependencies shift, but still lets it restore quickly on later runs when nothing changed. Also, the first run right after caching goes in is always a cache miss because there is no previous cache entry yet. After that though, runs start

benefiting from restoration. This detail was handled in the experimental design by doing 11 runs per cached configuration and leaving out the very first run from the analysis.

4.2 Job Parallelization Implementation

Parallelization strategies were put in place by remixing the workflow configurations a bit, so that independent tasks can run at the same time on different GitHub Actions runners. The main principle behind these changes was pretty simple: find the parts in a workflow that do not truly depend on each other, meaning tasks where neither one needs the output or the artefacts of the other. Then you turn them into separate jobs so they can execute in parallel rather than being stacked as step by step actions inside one single job, even if it kind of looks cleaner that way at first.

In stripe/stripe-python the original workflow already had those four separate jobs (lint , build , test , and publish) , but inside the lint job there was also a type-check thing that marched through seven Python minor versions, sort of one by one, using a shell loop. So each run called the type-checker for a specific Python version, and all seven of those checks were basically serialized on the same runner. That got flagged as a chance to parallelize , because every type-check run is independent. Like the output from Python 3.6 doesn't really influence the check against Python 3.7, and same idea for 3.8 and onward. The change was to pull that type-checking step out into its own dedicated typecheck job, and then use a matrix strategy that spans all seven Python minor versions. With that setup, the seven separate type-check operations can happen at the same time, each on its own runner, while the lint job got trimmed down so it only does linting plus formatting checks. Lastly, the publish job's dependency list was updated so it now waits for the new typecheck job in addition to the existing build , test , and lint jobs.

For nashtech-garage/yas, the original cart service workflow was kinda a single monolithic job i guess that just did everything in a straight line, like Maven build and test, then checkstyle analysis, test result reporting, SonarCloud analysis, OWASP dependency checking, coverage reporting, and finally Docker image building. With that setup, each operation had to patiently wait until all the previous ones finished. Even more, many of those steps were pretty independent, but the old structure still forced

them to be sequential, which feels not ideal. The modified workflow, instead, split the whole thing into five separate jobs that can run at the same time. The build-and-test job does the Maven compilation and unit testing, and it outputs build artefacts that get uploaded so later jobs can actually use them. Then the checkstyle job runs static code analysis on its own, no need to block on the other tasks. After that, the security-scan job performs SonarCloud analysis along with OWASP dependency checking, all independently too. There's also the coverage job, and it's configured to start only after build-and-test completes, because it needs the compiled test results. Same idea for the docker job: it depends on build-and-test, since it requires the compiled application. So in practice, the build-and-test, checkstyle, and security-scan jobs run concurrently. Meanwhile, once build-and-test is done, the coverage and docker jobs kick off immediately, even if checkstyle and security-scan are still doing their thing.

Parallelization was not applied to `django/djangoproject.com` or to `sparrowwallet/sparrow`, for reasons that are kinda tied to the specific way each project's workflow architecture is laid out. On `django/djangoproject.com`, the current workflow is basically one tightly coupled test job, and it leans on a PostgreSQL database service container. It also includes Playwright browser installation, then database schema creation, plus test data loading. These parts are prerequisites for the tests, so they can't really be broken apart in a way that stays clean and useful. If you tried to split the job into parallel components you'd end up duplicating the database service and all that associated setup work across multiple runners. That means a lot of redundancy, plus extra runner costs, and I do not see meaningful performance improvements coming out of it. In other words the workflow is already optimized in the sense that it does one cohesive task with all the needed infrastructure kept together on a single runner. For `sparrowwallet/sparrow`, parallelization is already being handled via its operating system matrix strategy. That approach builds the application concurrently for multiple platforms (Linux, macOS, and Windows). Within each individual platform build, no real extra parallelization opportunities were spotted, at least none that looked practical. And this outcome, that not every repository benefits from parallelization, is actually a useful

result on its own. It shows that whether an optimization strategy can be applied depends heavily on the particular shape of each workflow, not just on the idea itself.

4.3 Forking and Execution Methodology

Each of the four intervention repositories got forked over to the researcher's GitHub account `github.com/SagarGi`. The workflow files were edited straight on the fork's default branch, using the GitHub web editor, and workflow runs were kicked off via the `workflow_dispatch` event. This is nice because it lets you manually trigger from the Actions tab without actually pushing any new code. For anything that involved dependency caching, 11 runs were started per configuration. The first one is basically the cache warmup, so it's a cache miss, then the next 10 runs happen with the cache already there (so cache hits). For the configurations that had no caching, like parallelization-only, 10 runs were triggered in total. In the post-optimization analysis, only runs that ended successfully were taken into account.

During the forking run, there was a practical hiccup, namely that quite a few workflow steps really need repo specific secrets, and those are not carried over when you fork. In particular things like SonarCloud authentication tokens, container registry credentials, plus macOS code signing certificates just don't make it. So for the steps that were impacted, they were set up with `continue-on-error: true`, which basically lets a step go sideways without stopping the entire workflow. We picked this option rather than skipping those steps completely, because keeping the attempted authentication is what preserves the time already spent, and that way the total workflow length stays in the same neighborhood as in the original repository. The gap between a failed authentication and a successful one is tiny, usually under one second, and it's mostly just the server's refusal response that comes back fast. For `sparrowwallet/sparrow` there was one extra change too: the build matrix got cut down to Ubuntu only, since the macOS jobs kept failing because the needed code signing certificates are tied to the original repository's maintainer. This restriction is mentioned in the results analysis.

Stage 5: Results and Analysis

5.1 Baseline Performance

We set the baseline build performance by taking data from at least 30 of the most recent successful workflow runs per repository, straight from the original, unmodified projects. This was done through the GitHub Actions API and honestly we stuck pretty close to that number, because you need enough samples. At first we gathered 167 workflow runs across the five repositories.

Then came the cleaning part where we noticed eight records that looked weird. They showed durations over an hour, which felt off. When we dug in, it turned out they weren't really about compilation time or build execution at all. Instead, those outliers were workflow runs that basically sat in the queue for a long time because GitHub Actions' shared runner infrastructure was under heavy demand, so the "waiting" time got mixed into the duration. Because of that, we removed those eight entries and ended up with 159 valid observations for the baseline analysis.

Figure 1 shows the mean baseline build times for each repository, with standard deviation style error bars.

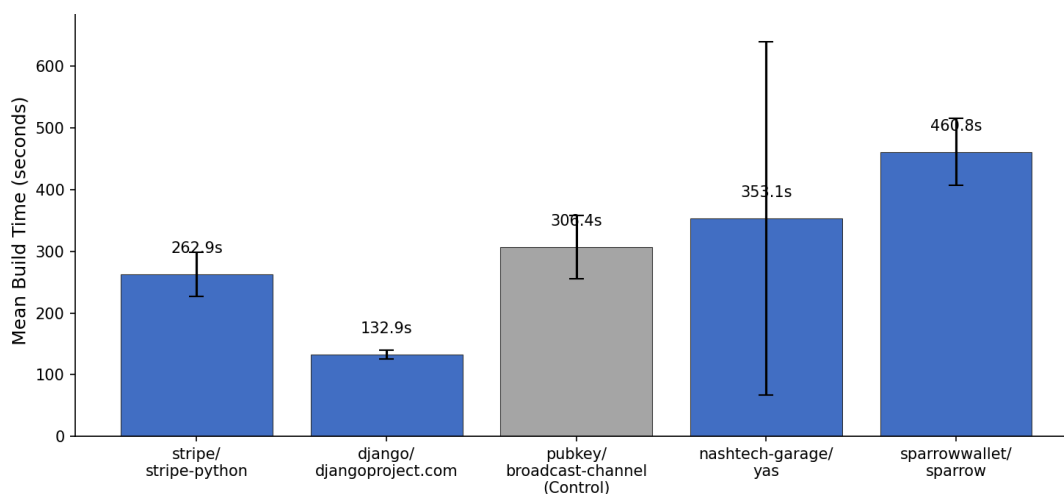


Figure 1: Baseline Build Times Across Selected Repositories

Those repositories show pretty different performance behavior, yeah, and it kind of tracks their architectures, how big the dependencies are, and the actual workflow setup.

For example `django/djangoproject.com` kind of stands out for having the shortest and also the more consistent builds, mean time comes in at 132.9 seconds, with a standard deviation of only 6.8 seconds. That usually means the pipeline is stable, and pretty predictable. Then `stripe/stripe-python` gives a mean of 262.9 seconds, but the spread is more noticeable, with moderate variability with $\sigma = 35.5$ seconds. Meanwhile `sparrowwallet/sparrow` is where things drag the most, it has the longest mean build time, about 460.8 seconds, and this seems linked to its multi-platform matrix builds across five operating system targets. `nashtech-garage/yas` has the highest variability, $\sigma = 286.4$ seconds, which lines up with the microservice workflows being more uneven or complex. Inside that, the inventory service averages 233.3 seconds, whereas the product service jumps way up to 518.2 seconds. And the control repo `pubkey/broadcast-channel`, which already does both caching and matrix builds, lands at a mean of 306.4 seconds with a standard deviation of 51.4 seconds. So it sits between the calmer builds and the more chaotic ones, more or less.

5.2 stripe/stripe-python Results

On the `stripe/stripe-python` repo, we tested three optimization setups, like caching-only, just parallelization, and a combined approach (not sure if both in the same run) . Figure 2 shows how each tactic changes things, relative to the baseline, so you can see the differences.

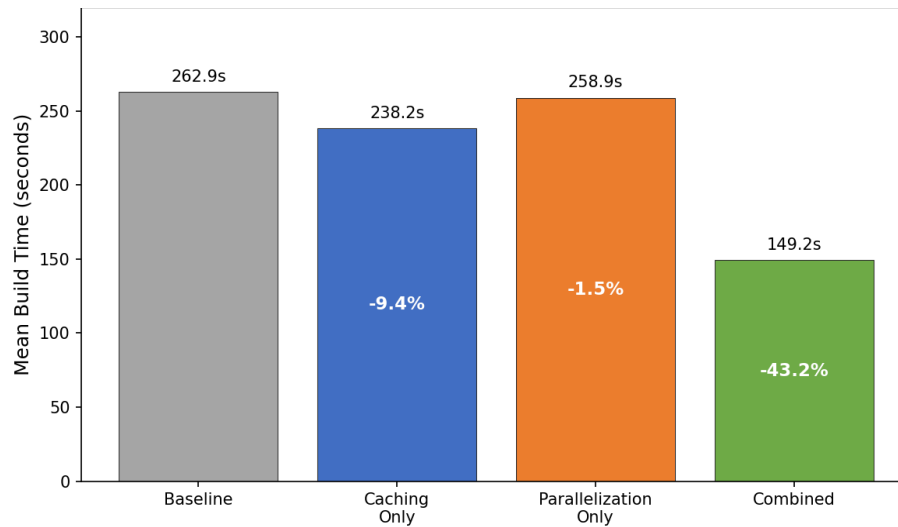


Figure 2: *stipe/stripe-python*- Optimization Impact

Caching by itself gave a fairly modest 9.4% drop in mean build time, going from 262.9 seconds to 238.2 seconds. This improvement is kinda small and, honestly, it makes more sense when you look at how *stripe/stripe-python* handles dependencies or more like how the whole dependency story plays out. The repo uses the just task runner, which creates and manages virtual environments on its own. So yes, the pip cache helps with pulling Python packages faster, but it still does not remove the extra time cost tied to creating those virtual environments and then installing the packages inside them. Also, the dependency set is relatively small, especially compared to typical enterprise Java projects, so the amount of time you can really claw back through caching is constrained.

Parallelization alone, on the other hand, only improved things by about 1.5%, with the mean falling to 258.9 seconds. Even though the type-check loop was successfully pulled into a matrix job that runs seven Python versions at the same time, each matrix cell still has to provision its own runner, plus all the setup work. In other words, the time you save by type checking in parallel gets mostly eaten up by the overhead of spinning up seven extra runners. Each one still has to check out the repository, set up Python, install the project's dependencies, and only then run the type checker.

So, the merged configuration kind of pulled off a pretty substantial 43.2% reduction, and the mean build time ended up at 149.2 seconds. That comes with a low standard deviation of only 6.2 seconds, while the baseline was up at 35.5 seconds. There's a clear synergistic thing going on here: the combined improvement at 43.2% is way larger than you'd expect if you just added up the separate gains ($9.4\% + 1.5\% = 10.9\%$). In other words, it's not merely "more of the same". The reason is the way the two strategies interact. Once caching is used together with parallelization, the per-runner setup time gets cut down because the cache takes over, so the extra overhead from provisioning several runner instances for the matrix job is partly softened, kind of reduced. Meanwhile the type-check runners, which before had been wasting time re-installing dependencies before each individual check, now restore those dependencies from cache. That change makes the parallelization advantage show up more clearly, instead of being swallowed by repeated setup. Overall, this outcome shows that using the optimization strategies together can be super-additive. The combined effect, bigger than the arithmetic total, has real, practical consequences for developers trying to choose what to implement first.

5.3 django/djangoproject.com Results

Dependency caching was basically the only thing we tried on `django/djangoproject.com`, and yeah the single job workflow layout didn't really give us any practical "split it up" parallelization options, like we mention in Section 4.2. But then, against what we expected, the cached configuration showed a mean build time of 151.0 seconds. That's 13.6% slower than the baseline mean of 132.9 seconds.

This kind of backwards outcome deserves a closer look. For this project, a few things, tied to its dependency profile, help explain why caching turned out negative. First, the workflow installs a package (`django-push`) directly from a Git repository by cloning a particular commit. Those Git-cloned packages can't be served from the pip cache, so the clone operation has to run end to end on every run, caching or not. Second, the Playwright browser installation, which is a big chunk of the workflow setup time, is only partly helped by caching. Sure the browser binaries can be cached, yet the installation process includes platform specific chores. Like, you still have to run steps such as

installing system libraries via apt-get every time. Third, there is also the cost of the cache action itself: computing the cache key hash, searching for a matching hit in GitHub's cache storage, then downloading and extracting the cache archive, and finally uploading any new or modified files back to the cache at the end. That's not free, it adds real time. So when the fraction of dependencies that actually benefit from caching is small compared to the whole setup time, the overhead can cancel out—or even surpass—the savings, giving you a net negative effect.

This finding is one of the more significant results of the research, like it shows that dependency caching isn't always, universally useful. The impact of caching really depends on what the project's dependency profile looks like—especially the share of dependencies that are installed through standard package managers and can thus be cached in a way that actually makes sense. If a project leans heavily on non-standard installation paths, or on runtime specific setup steps, or even system level package installations then cache management overhead can end up being bigger than the time savings, even if you expect the opposite.

5.4 nashtech-garage/yas Results

The nashtech-garage/yas repo, kind of tested thru its cart service workflow, gave the clearest optimization results across all three configurations. In Figure 3, you can see the comparative performance data.

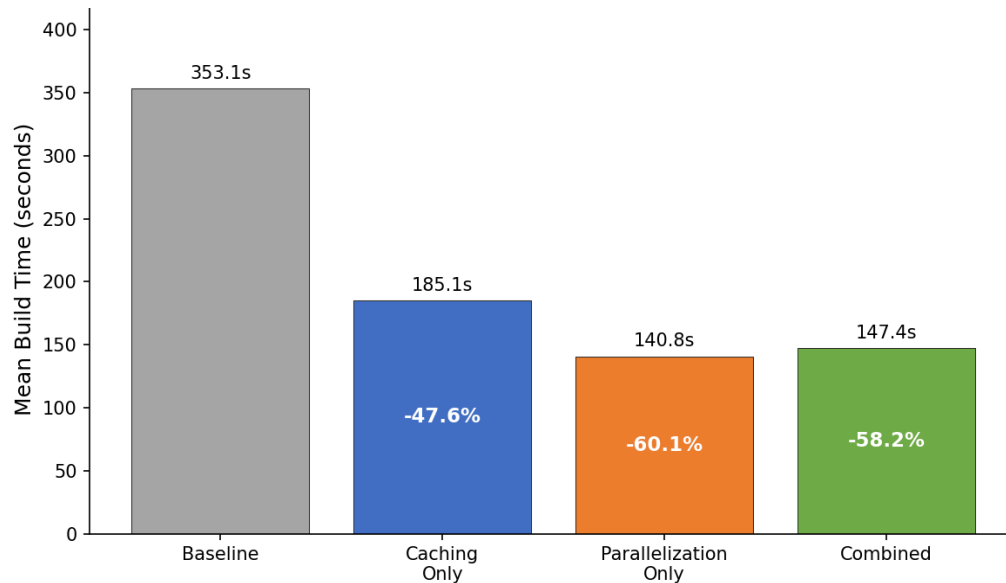


Figure 3: nashatech-garage/yas - Optimization Impact

Caching alone cut the average build duration by about 47.6%, from 353.1 s down to 185.1 s. At the same time the spread got way tighter, standard deviation went to 14.9 s, versus 286.4 s in the baseline, so, it's pretty noticeable. This kind of step-change usually shows up when Maven based Java projects lean hard on dependency resolution. If caching wasn't used, each workflow run re-downloads the whole dependency graph, basically hundreds of JAR files that come from Maven Central plus whatever other repositories are configured, then it checks their checksums and writes everything into the local repository again. Once the `~/.m2/repository` folder is cached, those already fetched artifacts just get put back from the cache in one shot. That means the nasty parts of a cold resolve—network latency, raw download time, and the checksum verification overhead—don't really happen in the same way anymore. You also see the same story in reliability: consistency improved from $\sigma = 286.4s$ to $\sigma = 14.9s$, so a big chunk of the earlier variability was tied to network conditions and dependency download timing. Caching removes that particular source of fluctuation almost entirely.

So, just parallelization alone ends up giving an even bigger reduction, about 60.1%, and the mean build time drops to 140.8 seconds, with a standard deviation of 8.6 seconds only. This kind of improvement pretty much comes from the monolithic build

job being reworked into five jobs running at the same time. Before, in the original setup, Maven compilation, checkstyle analysis, SonarCloud scanning, OWASP dependency checking, and Docker image building all went one after another inside a single job. So every step was forced to wait, right until the earlier ones finished. After the change, the build-and-test, checkstyle, and security-scan jobs run at the same time on different runners, and the total workflow length is basically the duration of the longest of those concurrent jobs, not the total sum of everything stacked in sequence.

Now, the mixed or combined configuration still performs well, but it gives a 58.2% reduction (mean 147.4 seconds). That is slightly weaker than the parallelization-only case at 60.1% , and at first it feels a bit backwards. The main reason seems to be the extra overhead from managing cache items across multiple parallel jobs. With the combined approach, each of the three concurrent jobs (build-and-test, checkstyle, and security-scan) keeps its own Maven cache entry. Then each job has to do cache key computation, download the cache, and later upload it again. Since these jobs are running in parallel, that cache maintenance overhead shows up on each runner simultaneously, and it doesn't get "paid back" by time savings as strongly as when only one job is relying on caching. Still, the gap between the two results is only 1.9 percentage points, which suggests the cache overhead is small, but not zero, and definitely measurable.

5.5 sparrowwallet/sparrow Results

Dependency caching was basically the only thing we optimized in sparrowwallet/sparrow, and it was aimed at the Gradle cache plus the wrapper directories. The cached configuration ended up recording a mean build time of 439.6 seconds , compared to the baseline mean of 460.8 seconds, so that's like a modest 4.6% gain. Also, the standard deviation dropped in the cached runs (22.1 seconds versus 54.0 seconds in the baseline) which suggest caching makes builds steadier, even if the absolute time improvement feels small, probably because it removes some of the randomness that comes from network conditions while dependencies are being fetched

One caveat worth saying here, is that this comparison is kinda apples to oranges. The baseline measurements came from the original repo workflow which builds across five operating system targets (Windows, two Ubuntu variants and two macOS variants) using a matrix, so lots of environment differences exist. After the optimization, the runs were done on Ubuntu only, since the macOS jobs failed because of missing code signing certificates tied to the original maintainer's Apple Developer account, and those certs are simply not present in the forked repository. So, that 4.6% number is really just the caching benefit for one platform build, not a clean match against the multi platform baseline. Still, even with that, the data shows Gradle dependency caching gives a measurable improvement in build times, even if it's not huge, and it lines up with what we saw in the other repositories, where caching mainly helped with dependencies that are managed by more normal package manager flows.

5.6 Cross-Repository Comparison

Figure 4 kinda shows the impact of dependency caching across all four intervention repos , so you can directly compare how well caching works in different languages and project types. Figure 5 then goes on to sort of summarize the optimization outcomes in a more full manner, like a complete overview of everything that improved..

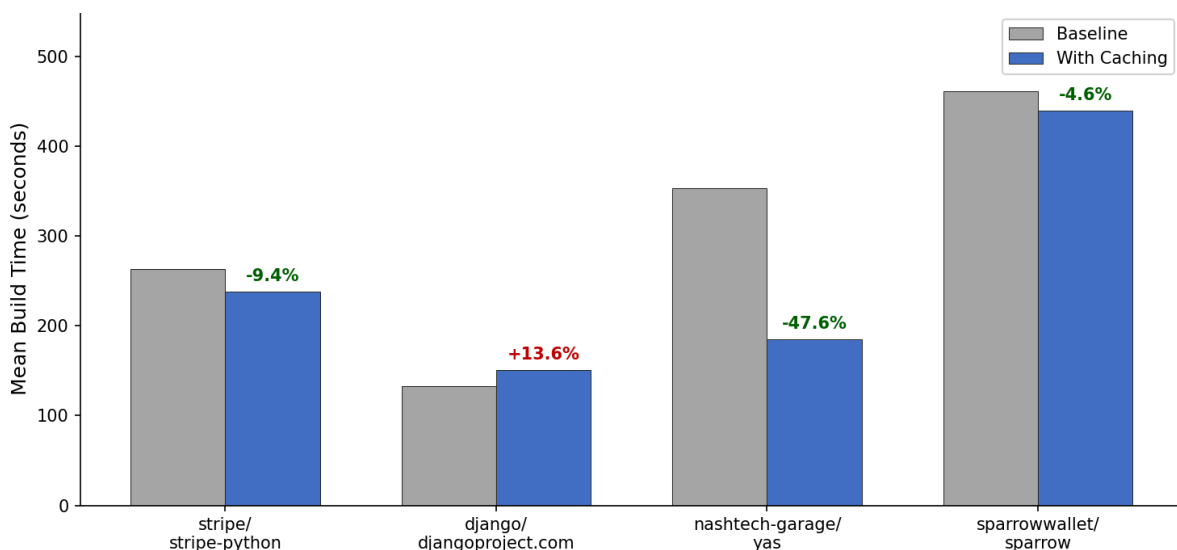


Figure 4: Effect of Dependency Caching Across All Repositories

Figure 5: Summary of Build Time Reductions Across All Repositories

Repository	Caching	Parallelization	Combined	Best Result
stripe/stripe-python	-9.4%	-1.5%	-43.2%	-43.2% (Combined)
django/djangoproject.com	+13.6%	N/A	N/A	No improvement
nashtech-garage/yas	-47.6%	-60.1%	-58.2%	-60.1% (Parallel)
sparrowwallet/sparrow	-4.6%	N/A	N/A	-4.6% (Caching)

Figure 5: Summary of Build Time Reduction Across All Repositories

Table 4: Complete Results Summary

Repository	Baseline(s)	Caching(s)	Cache Δ	Parallel(s)	Parallel Δ	Combined(s)
stripe/stripe-python	262.9	238.2	-9.4%	258.9	-1.5%	149.2 (-43.2%)
django/djangoproject.com	132.9	151.0	+13.6%	N/A	N/A	N/A
nashtech-garage/yas	353.1	185.1	-47.6%	140.8	-60.1%	147.4 (-58.2%)
sparrowwallet/sparrow	460.8	439.6	-4.6%*	N/A	N/A	N/A

*** Baseline measured with 5-OS matrix; post-optimization measured on Ubuntu only.**

When we look across repositories it kind of shows a few patterns about how good build time optimization really is. Dependency caching especially, changes a lot based on the project's own dependency profile. In some repos, for example ones with huge dependency trees that they resolve from remote package repositories, like Maven used in nashtech-garage/yas, caching landed around a 47.6% reduction. But in other cases, if the project only has a small set of dependencies or the dependencies come from

non-standard places , like `django/djangoproject.com` which ended up with a 13.6% increase , caching might do nothing useful, or it can actually make things slightly worse. The main thing that seems to decide it, is how much of the overall setup time is really coming from dependency downloads that are cacheable.

Second, parallelization is most effective when the original workflow has some monolithic job that ends up doing several independent tasks one after the other, like it just keeps going even though it could be split. `nashtech-garage/yas`'s monolithic build job, once restructured into five concurrent jobs , showed a 60.1% improvement , which was also the biggest single gain in this research. On the other hand , when the jobs are already set up as independent parallel pieces (like `sparrowwallet/sparrow`'s OS matrix) or when the workflow is basically tightly coupled steps that cannot be split in any meaningful way (like `django/djangoproject.com`) , then the chances for more parallelism become thin or just not there at all.

Third, the combined outcome of caching and parallelization is not really additive in any straightforward sense. The way they interact depends on the project's internal shape, basically. For `stripe/stripe-python`, the combined setup created a synergistic effect: 43.2% combined improvement was noticeably larger than the sum of the separate effects ($9.4\% + 1.5\% = 10.9\%$). The reason seems to be that caching trimmed per-runner setup time enough so the parallelization wins could actually show up. For `nashtech-garage/yas` though , the combined configuration (58.2%) landed a bit under parallelization by itself (60.1%). Here, the extra overhead from managing separate cache entries across multiple parallel jobs apparently ate into the caching benefit, more or less. Overall, these contrasts point to why you really need empirical testing: the interaction between optimization strategies can't be trusted from first principles, and it has to be measured for each specific workflow configuration , because the behavior is context dependent.

5.7 Answering the Research Questions

RQ1: To what extent does adding dependency caching and job parallelization reduce build times in GitHub Actions workflows, separately and also together?

The empirical outcomes show that the effect isn't consistent, it depends quite a lot on what kind of project it is, and on how the CI/CD pipeline is put together. For dependency caching, build times went down by something like 4.6% to 47.6% in three out of the four repositories we checked, and the biggest wins showed up where the dependency graph was big, and it was mostly handled through usual tools like Maven and Gradle. Still, in one repository caching backfired, it was slower by about 13.6%. That suggests the dependency profile there just wasn't friendly to caching, so it's not a guaranteed improvement, not at all.

Job parallelization seems to help too, but only "where applicable", it reduced build times by 1.5% up to 60.1%. The strongest improvements were again in places where a single monolithic job got broken into multiple concurrent tasks, so parallel steps could actually overlap rather than just repeat. When both dependency caching and job parallelization were used at the same time, we saw reductions of 43.2% and 58.2% in the two repositories where both approaches fit. And the combined result hints at an interaction effect—sometimes synergistic, sometimes a bit subtractive—depending on the overall workflow architecture.

RQ2: What practical guidelines can be derived from the empirical findings to help developers optimise their CI/CD pipelines?

The findings back up a set of evidence based recs, even if they're a bit context dependent. First, dependency caching should be put upfront for projects with big dependency trees that are handled by the usual package managers (pip, Maven, Gradle, npm) where the lock file basically stays put from build to build. But developers should be careful about caching if dependencies come from non standard sources like Git repositories, or when a lot of setup time is spent on steps that caching will not speed up, such as browser installation or database provisioning. Next, job parallelization should be on the table whenever a workflow has just one job doing several independent tasks one after another. In that case, developers should audit workflow configs for

sequential steps that do not actually rely on the same intermediate outputs, or anything that looks like a real data dependency. Also, if developers use both strategies at once, they should remember that separate cache entries across parallel jobs can add overhead, which might shrink the caching gain a little. So the combined effect should be checked with real measurements rather than treated as automatically additive. Overall, the strongest angle is to treat optimisation as an iterative thing—apply a change, measure carefully, then keep only what shows clear improvement. And you should run enough times to handle runner variability, since otherwise the results can feel... not exactly trustworthy.

6. Limitations

This research, while providing empirical evidence about how build time optimization strategies in GitHub Actions work, still has some limitations that really should be kept in mind when you're trying to interpret the results.

The biggest issue is the sample size, sort of, because the study only looked at five repositories. Out of those five, four were used as intervention targets and one repository acted as a control. That said, this sample looked enough to do a detailed case-study style analysis and it also helped spot meaningful patterns across different languages and types of projects, but it still narrows how far the findings can be generalized to the wider population of GitHub Actions users. Also, the repositories weren't chosen randomly from the whole ecosystem. They were selected from a pool of mid-sized open-source projects with roughly 100 to 2,000 stars, so the outcomes may not carry over neatly to enterprise-scale environments where CI/CD pipelines might be organized very differently, or where proprietary tooling is common, or where self-hosted runner infrastructure changes the whole setup. A larger sample, covering more project scales, different industries, and a broader mix of workflow intricacies, would likely strengthen the external validity of what the study claims.

The number of post-optimization runs per configuration is kind of a constraint as well. Each optimization setup was tried with 10 successful runs, which was picked to kinda balance statistical validity against the real-world limits of GitHub Actions, especially the resource consumption part. With 10 runs you can usually still spot big shifts—like the 47.6% caching improvement seen in `nashtech-garage/yas`—but the confidence intervals end up wider than they would with a bigger sample, so the statistical estimates become less crisp. This matters even more for the smaller observed effects, for example `stripe/stripe-python` with a 9.4% caching improvement and `sparrowwallet/sparrow` with only a 4.6% improvement. In those cases the uncertainty margin is relatively larger compared to the effect size that's being reported, so the results can feel more noisy even if the direction is consistent.

There's also a very specific limitation for the sparrowwallet/sparrow comparison. The baseline numbers were taken from the original repo's workflow, and that workflow builds for five operating system targets using a matrix strategy. In the forked repository, macOS code signing certificates were not available, so the post-optimization runs had to be performed on Ubuntu only. Because of that, baseline and post-optimization durations can not really be compared directly, since the baseline duration includes the elapsed time until the slowest of five concurrent platform builds finishes, while the post-optimization duration corresponds to a single-platform build. So the 4.6% caching improvement reported for this repository should be treated as a hint of the per-platform benefit, not as a strict, like-for-like comparison.

Using GitHub Actions shared runner infrastructure kind of adds a level of variability you just can not fully steer. Build times on shared runners end up being nudged by things that live outside the workflow itself, like runner provisioning delays, network conditions that slow down dependency downloads, plus whatever the current load is on GitHub's side. You could actually see this in the baseline set, where eight runs had to be removed because the queuing delays went past one hour. Then later, in the post-optimization data, there were also some occasional odd outliers that showed up, kind of randomly. Even though doing multiple runs for each configuration helps smooth out the impact of any single weird run, the real issue is that the core variability is still there, so small differences between configurations should be read carefully, not too confidently.

Lastly, the research only zoomed in on two optimization approaches: dependency caching and job parallelization. There are other methods that might also cut down build time, for example incremental builds, selective test execution, workflow-level concurrency controls, and using self-hosted runners, but those were out of the scope for this work. Going forward, future research could expand the whole method to test these other strategies too, including how they might play together with the two techniques we looked at here.

7. Conclusion

This research set out to do an empirical analysis of build time optimization strategies in GitHub Actions workflows, with a specific focus on checking the measurable impact of dependency caching and job parallelization, kind of at the same time. Through a systematic six-stage methodology—repository selection, baseline data collection, inefficiency identification, optimization implementation, post-optimization measurement, and cross-repository analysis—the study looked at five open-source repositories that span three programming languages and different project architectures. The results actually offer quantitative evidence that answers both research questions and adds to the growing body of literature on CI/CD pipeline optimization , even if things vary across contexts.

The first research question was basically asking how much the introduction of dependency caching and job parallelization really cuts down build times in GitHub Actions workflows , both when you look at them on their own and when you combine them together . The empirical results show that both approaches can lead to big improvements, but how well they work seems very dependent on the concrete traits of the project and the exact workflow setup. For example, dependency caching lowered build times by 47.6% in the Maven-based nashtech-garage/yas repository, 9.4% in stripe/stripe-python, and 4.6% in sparrowwallet/sparrow. These findings seem to back the idea that caching works best when the project has a large dependency graph , especially when those dependencies are handled by the usual package managers, since a noticeable chunk of the build time is spent fetching and resolving artifacts from remote endpoints. Still, the django/djangoproject.com case was a clear exception , where caching actually made things worse by 13.6%. That negative result likely came from the fact that the project relies on rather non-standard dependency sources, such as Git-cloned packages and platform-specific browser installs. Those components just cannot be properly reused from cache, and on top of that, the overhead tied to cache management ended up being larger than the small gains on the parts that were actually cacheable .

The parallelization of the job stuff brought the biggest jump in this study, cutting build times by 60.1% in nashtech-garage/yas, once the original monolithic, single job flow got reworked into five concurrent jobs. Like, this result basically points out a clear performance penalty from sequential workflow architectures where independent tasks get forced to happen one after another, and it also shows how much you can gain when those tasks are spotted and changed so they run concurrently. Meanwhile in stripe/stripe-python parallelization by itself only nudged things up by 1.5% , because the cost of setting up extra runners for the pulled out type-check matrix job pretty much ate up most of the time that concurrent execution would have saved. So yeah, these two repo patterns together suggest that parallelization helps the most when the workflow has tasks that are genuinely independent and also last long enough to matter, not when you slice up smaller, lighter operations into separate jobs and call it a day.

Putting both strategies together showed something that actually matters in practice, about how caching and parallelization interact. In stripe/stripe-python, when we ran the combined setup it sort of clicked, giving a 43.2% drop which was noticeably more than the separate gains added up ($9.4\% + 1.5\% = 10.9\%$). That kind of super-additive behavior suggests caching can basically make parallelization “stick” better, because it trims the per-runner setup overhead, which otherwise erodes the value of launching multiple concurrent jobs. On the flip side, in nashtech-garage/yas the combined configuration did not shine as much. There, we saw a 58.2% reduction but parallelization by itself was 60.1% so the combo was marginally worse. It looks like the overhead from keeping separate cache entries across multiple parallel jobs, can slightly eat into the caching advantage. Overall, these two cases really bring home the point that the combined influence of optimization strategies can’t be guessed reliably just from their standalone effects, you have to verify it by measurement for each particular workflow, not just infer it.

The second research question was basically, what sort of practical directions can you pull out from all the findings. From the empirical evidence, there are a few recommendations you can actually try. For one, developers should really push dependency caching first for projects that lean on usual package managers like Maven,

Gradle, pip, and npm; especially when the dependency hierarchy is large and the thing basically stays the same across builds. Before turning caching on, they should check how much of their workflow setup time is truly spent on dependency downloads that are even cacheable, if that part is not big then the extra burden of cache handling might wipe out the gains. Also job parallelization, it should be on the table whenever you see a single job that quietly does multiple independent activities one after another, and developers should scan their pipeline setup to spot steps that do not share any input state, so they can run at the same time. If you apply both approaches together, don't just take the results of each method separately and add them up like it's guaranteed, instead measure the combined impact directly, because the interaction could be helpful in a synergistic way or maybe a little subtractive depending on how the workflow is arranged.

The research also kind of comes with a methodological framework that others can take, replicate , and maybe extend later on by future researchers. This six stage approach, starting with automated repository selection, and then ending with the statistical comparison of before and after measurements, gives you a clear and repeatable procedure for testing CI/CD optimization strategies. The Python scripts that were made for repository selection, baseline data collection, and then the post optimization data collection are reusable building blocks, so other teams can adapt them for comparable investigations. Also, the choice to include a control repository, which already implements both optimization strategies, turned out to be quite useful, because it offers a kind of comparative reference point . So, this practice should be used again in future studies as well.

A few paths for future work kind of jump out from this study. For starters the sample size could be widened to pull in a bigger and more varied set of repositories, maybe even enterprise-scale projects, with self hosted runner infrastructure involved, just to see if the results really hold up outside of open-source settings that rely on shared runners. Next, the approach could be pushed further to look at optimization tactics beyond caching and parallelization, like incremental builds, targeted test runs that trigger only on changed files, workflow level concurrency knobs, and container based build environments.

It would also be useful to do a longer term check, like a longitudinal study following the day to day and week to week effects after optimizations are actually rolled out , over periods of weeks or months. That would clarify if the gains stay stable especially when you start worrying about cache invalidation rates and whether the extra maintenance effort from complicated parallelized workflow layouts becomes a headache. Finally, a cost versus benefit analysis would make the argument much more concrete, by estimating the financial savings tied to reducing build times on cloud CI infrastructure. That sort of quantification would help teams decide which improvements to prioritize first, and not just chase whatever seems fastest on paper.

In conclusion, this research sort of shows that dependency caching along with job parallelization can really cut down build times in GitHub Actions workflows, yet the impact is not always the same or it is never fully guaranteed. The best gains showed up where the dependency trees were huge, and also when the workflow setups were more monolithic, like all in one place, you know. On the other hand projects that had already been tuned pretty well, or those with unconventional dependency profiles, tended to see smaller improvements—or sometimes nothing noticeable at all. Also, the fact that using both strategies together can turn out super-additive is a strong, practical reason to try implementing them side by side, not separately. Still, there's that counter-example where caching somehow caused slower performance, so it feels like a warning not to apply optimizations broadly without checking what happens. Overall, these findings give developers evidence based direction for deciding which strategy or strategies to pick, and for measuring whether it actually helps in their own repositories, rather than assuming it will.

8. Future Work

From the findings in this study, a few lines of future research kind of come up, not in one neat list but more like in layers. For starters, the range of optimization strategies we looked at can be broadened. Right now it's mostly dependency caching and job parallelization, but there's room for other options like incremental builds, running only the tests that match the changed files, Docker layer caching, and even relying on self-hosted runners. Testing these add-ons with the same empirical methodology would give a wider view of what's actually available to GitHub Actions users, and which method tends to win in practice. Second, the sample size could use some expansion. We could include more repositories that are also more varied—think enterprise scale systems, closed source codebases, and projects that use other CI/CD platforms such as GitLab CI or CircleCI. Doing this would likely strengthen how general these results are, and it would make it possible to compare optimization effectiveness across platforms instead of staying within one ecosystem. Third, a longitudinal approach would be really useful. Instead of just measuring performance at one point, we could track what happens after optimizations are adopted over a longer stretch of time. In particular, watching cache invalidation rates over the months would help clarify how often dependency updates make cached material stale and whether the long-term average gain stays similar to the short-term improvement we observed here. Fourth, we could add a more formal cost-benefit analysis. If we put numbers on the financial savings tied to shorter build times it would make a stronger argument for why teams should invest in these optimizations, beyond just the technical performance angle.

Since GitHub Actions bills usage per minute for private repos, even small, percentage-ish improvements can end up meaningfully lowering costs at scale, kinda like a compounding thing. If we can quantify how this relationship actually behaves, it would let dev teams choose where to focus their optimization work, instead of just guessing. And then, building an automated tool, or a GitHub Action really, that reads a repository's workflow config and suggests practical optimization strategies based on the patterns we found in this research would be a useful, hands-on contribution. That way

the results become directly usable by developers, without them having to run all their own empirical analysis.

9. References

Bouzenia, I. and Pradel, M. (2024) 'Resource Usage and Optimization Opportunities in Workflows of GitHub Actions', *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. IEEE. Available at: <https://doi.org/10.1145/3597503.3623303>

Decan, A., Mens, T., Rostami Mazrae, P. and Wessel, M. (2022) 'On the evolution of GitHub Actions workflows', *Empirical Software Engineering*. Available at: <https://arxiv.org/abs/2207.00466>

Faqih, A.R., Taufiqurrahman, A., Husen, J.H. and Sabariah, M.K. (2024) 'Empirical Analysis of CI/CD Tools Usage in GitHub Actions Workflows', *Journal of Informatics and Web Engineering*, 3(2), pp. 241-255. Available at: <https://doi.org/10.33093/jiwe.2024.3.2.16>

Gallaba, K., Ewart, J., Junqueira, Y. and McIntosh, S. (2022) 'Accelerating Continuous Integration by Caching Environments and Inferring Dependencies', *IEEE Transactions on Software Engineering*, 48(6), pp. 2040-2052. Available at: <https://doi.org/10.1109/TSE.2020.3048335>

Gallaba, K. and McIntosh, S. (2020) 'Use and Misuse of Continuous Integration Features: An Empirical Study of Projects That (Mis)Use Travis CI', *IEEE Transactions on Software Engineering*, 46(1), pp. 33-50. Available at: <https://doi.org/10.1109/TSE.2018.2871560>

Gallaba, K., Lamothe, M. and McIntosh, S. (2022) 'Lessons from Eight Years of Operational Data from a Continuous Integration Service: An Exploratory Case Study of CircleCI', *Proceedings of the 44th International Conference on Software Engineering (ICSE '22)*, pp. 1330-1342. Available at: <https://doi.org/10.1145/3510003.3510212>

Ghaleb, T.A., Hassan, S. and Zou, Y. (2024) 'An Empirical Study of Complexity, Heterogeneity, and Compliance of GitHub Actions Workflows', *arXiv preprint*. Available at: <https://arxiv.org/abs/2507.18062>

Rostami Mazrae, P., Mens, T., Golzadeh, M. and Decan, A. (2023) 'On the usage, co-usage and migration of CI/CD tools: A qualitative analysis', *Empirical Software Engineering*, 28(2), Article 52. Available at: <https://doi.org/10.1007/s10664-022-10285-5>

Shahin, M., Babar, M.A. and Zhu, L. (2017) 'Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices', *IEEE Access*, 5, pp. 3909-3943. Available at: <https://doi.org/10.1109/ACCESS.2017.2685629>

Yin, M., Kashiwa, Y., Gallaba, K., Alfadel, M., Kamei, Y. and McIntosh, S. (2024) 'Developer-Applied Accelerations in Continuous Integration: A Detection Approach and Catalog of Patterns', *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*, pp. 1655-1666. Available at: <https://doi.org/10.1145/3691620.3695533>

GitHub (2024) *Caching dependencies to speed up workflows*. Available at: <https://docs.github.com/en/actions/using-workflows/caching-dependencies-to-speed-up-workflows> (Accessed: 28 April 2026).

GitHub (2024) *Using a matrix for your jobs*. Available at: <https://docs.github.com/en/actions/using-jobs/using-a-matrix-for-your-jobs> (Accessed: 28 April 2026).

10. Appendix

Appendix A: Github Links

All scripts, data collection tools, and optimized workflow configurations developed for this research are publicly available on GitHub:

https://github.com/SagarGi/westlondon/tree/main/my_dissertation/Scripts

The following links provide access to the GitHub Actions workflow runs executed during the optimization experiments on the forked repositories:

stripe/stripe-python: <https://github.com/SagarGi/stripe-python/actions>

django/djangoproject.com: <https://github.com/SagarGi/djangoproject.com/actions>

nashtech-garage/yas: <https://github.com/SagarGi/yas/actions>

sparrowwallet/sparrow: <https://github.com/SagarGi/sparrow/actions>

The original unmodified repositories used as the basis for this research are available at:

<https://github.com/stripe/stripe-python>

<https://github.com/django/djangoproject.com>

<https://github.com/pubkey/broadcast-channel>

<https://github.com/nashtech-garage/yas>

<https://github.com/sparrowwallet/sparrow>

Appendix B: Repository Selection Script

This script queries the GitHub Search API to identify candidate repositories matching the selection criteria defined in Stage 1 of the methodology.

Filename: stage1_find_repos_v2.py

```
import csv
import time
from datetime import datetime, timedelta
from github import Github, Auth

GITHUB_TOKEN = "YOUR_TOKEN_HERE"

MIN_STARS = 100
MAX_STARS = 2000
MIN_WORKFLOW_RUNS = 50
```

```

LANGUAGES = ["Python", "JavaScript", "Java"]
REPOS_PER_LANGUAGE = 10

def get_rate_remaining(g):
    try:
        rate = g.get_rate_limit()
        if hasattr(rate, 'core'):
            return rate.core.remaining
        else:
            return rate.rate.remaining
    except Exception:
        return 999

def check_has_caching(workflow_contents):
    content_lower = workflow_contents.lower()
    return "actions/cache" in content_lower or "cache:" in content_lower

def check_has_parallelization(workflow_contents):
    content_lower = workflow_contents.lower()
    job_count = content_lower.count("jobs:")
    has_matrix = "matrix" in content_lower
    return job_count > 0, has_matrix

def main():
    auth = Auth.Token(GITHUB_TOKEN)
    g = Github(auth=auth)

    print(f"Authenticated as: {g.get_user().login}")
    print(f"Rate limit remaining: {get_rate_remaining(g)}")

    six_months_ago = datetime.now() - timedelta(days=180)
    results = []

    for lang in LANGUAGES:
        print(f"\nSearching for {lang} repositories ({MIN_STARS}-{MAX_STARS}
stars)...")

        query = f"language:{lang} stars:{MIN_STARS}..{MAX_STARS}
pushed:>{six_months_ago.strftime('%Y-%m-%d')}"
        repos = g.search_repositories(query=query, sort="stars", order="desc")

        count = 0
        checked = 0
        for repo in repos:
            if count >= REPOS_PER_LANGUAGE:
                break
            if checked >= 60:
                break
            checked += 1

            try:
                print(f"Checking: {repo.full_name} ({repo.stargazers_count}
stars)...")

                try:
                    workflows = list(repo.get_workflows())
                except Exception:
                    continue

                if len(workflows) == 0:
                    continue

```

```

try:
    runs = repo.get_workflow_runs()
    total_runs = runs.totalCount
except Exception:
    total_runs = 0

if total_runs < MIN_WORKFLOW_RUNS:
    continue

try:
    workflow_files = repo.get_contents(".github/workflows")
    yml_files = [f for f in workflow_files if
f.name.endswith(('.yaml', '.yml'))]
    yml_names = [f.name for f in yml_files]
except Exception:
    yml_files = []
    yml_names = []

if len(yml_files) == 0:
    continue

uses_caching = False
has_multiple_jobs = False
has_matrix = False

try:
    for wf in yml_files[:3]:
        content = wf.decoded_content.decode('utf-8')
        if check_has_caching(content):
            uses_caching = True
        jobs_check, matrix_check =
check_has_parallelization(content)
        if jobs_check:
            has_multiple_jobs = True
        if matrix_check:
            has_matrix = True
except Exception:
    pass

recent_runs = []
try:
    for run in repo.get_workflow_runs(status="completed"):
        if run.conclusion == "success" and len(recent_runs) < 5:
            duration = (run.updated_at -
run.created_at).total_seconds()
            recent_runs.append(duration)
            if len(recent_runs) >= 5:
                break
except Exception:
    pass

avg_duration = sum(recent_runs) / len(recent_runs) if
recent_runs else 0

if avg_duration < 10 and avg_duration > 0:
    continue

repo_info = {
    "name": repo.full_name,
    "language": lang,

```

```

        "stars": repo.stargazers_count,
        "total_workflow_runs": total_runs,
        "workflow_count": len(workflows),
        "workflow_files": ", ".join(yml_names[:5]),
        "uses_caching": "Yes" if uses_caching else "No",
        "has_matrix_builds": "Yes" if has_matrix else "No",
        "last_pushed": repo.pushed_at.strftime("%Y-%m-%d"),
        "avg_build_time_seconds": round(avg_duration, 1),
        "url": repo.html_url,
        "description": (repo.description or "")[:100]
    }

    results.append(repo_info)
    count += 1

    remaining = get_rate_remaining(g)
    if remaining < 100:
        print(f"Rate limit low ({remaining}), waiting 60
seconds...")
        time.sleep(60)

    except Exception as e:
        print(f"Error: {e}")
        continue

    output_file = "stage1_candidate_repos_v2.csv"
    if results:
        with open(output_file, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=results[0].keys())
            writer.writeheader()
            writer.writerows(results)

        print(f"\nTotal candidate repositories found: {len(results)}")
        for lang in LANGUAGES:
            lang_repos = [r for r in results if r["language"] == lang]
            print(f"\n{lang}: {len(lang_repos)} repos found")
            for r in lang_repos:
                print(f"  - {r['name']} | {r['stars']} stars |
{r['total_workflow_runs']} runs | avg build: {r['avg_build_time_seconds']}s")

            print(f"\nResults saved to: {output_file}")

if __name__ == "__main__":
    main()

```

Appendix C: Workflow Verification Script

This script downloads and parses YAML workflow files from each selected repository to verify the presence of caching mechanisms, matrix strategies, and sequential dependencies.

Filename: stage1b_verify_workflows.py

```

import os
import yaml
from github import Github, Auth

```

```

GITHUB_TOKEN = "YOUR_TOKEN_HERE"

REPOS = [
    "stripe/stripe-python",
    "django/djangoproject.com",
    "pubkey/broadcast-channel",
    "nashtech-garage/yas",
    "sparrowwallet/sparrow",
]

def analyse_workflow(content, filename):
    try:
        wf = yaml.safe_load(content)
    except Exception as e:
        return {"error": str(e)}

    if not wf or "jobs" not in wf:
        return {"error": "No jobs found"}

    jobs = wf.get("jobs", {})
    analysis = {
        "filename": filename,
        "job_count": len(jobs),
        "jobs": {},
        "has_caching": False,
        "has_matrix": False,
        "has_sequential_deps": False,
        "caching_opportunities": [],
        "parallelization_opportunities": [],
    }

    for job_name, job_config in jobs.items():
        if not isinstance(job_config, dict):
            continue

        job_info = {
            "runs_on": job_config.get("runs-on", "unknown"),
            "needs": job_config.get("needs", []),
            "has_matrix": "matrix" in str(job_config.get("strategy", {})),
            "steps": [],
            "uses_cache": False,
            "installs_deps": False,
        }

        if job_info["needs"]:
            analysis["has_sequential_deps"] = True

        if job_info["has_matrix"]:
            analysis["has_matrix"] = True

        steps = job_config.get("steps", [])
        for step in steps:
            if not isinstance(step, dict):
                continue

            step_name = step.get("name", step.get("uses", "unnamed"))
            uses = step.get("uses", "")
            run = step.get("run", "")

            if "actions/cache" in uses:

```

```

        job_info["uses_cache"] = True
        analysis["has_caching"] = True

        if "cache" in str(step).lower() and ("actions/cache" in uses or
"cache:" in str(step)):
            job_info["uses_cache"] = True
            analysis["has_caching"] = True

    dep_keywords = ["pip install", "npm install", "npm ci", "yarn
install",
                    "mvn ", "gradle ", "maven", "composer install",
                    "bundle install", "go mod download"]
    for kw in dep_keywords:
        if kw in run.lower():
            job_info["installs_deps"] = True
            if not job_info["uses_cache"]:
                analysis["caching_opportunities"].append(
                    f"Job '{job_name}': runs '{kw}' without caching"
                )

        job_info["steps"].append(step_name[:60])

    analysis["jobs"][job_name] = job_info

    if analysis["has_sequential_deps"]:
        for job_name, job_info in analysis["jobs"].items():
            needs = job_info["needs"]
            if needs:
                needs_list = needs if isinstance(needs, list) else [needs]
                analysis["parallelization_opportunities"].append(
                    f"Job '{job_name}' depends on {needs_list}"
                )

    if not analysis["has_matrix"]:
        for job_name, job_info in analysis["jobs"].items():
            analysis["parallelization_opportunities"].append(
                f"Job '{job_name}' has no matrix strategy"
            )

    return analysis

def main():
    auth = Auth.Token(GITHUB_TOKEN)
    g = Github(auth=auth)

    output_dir = "workflow_analysis"
    os.makedirs(output_dir, exist_ok=True)

    for repo_name in REPOS:
        print(f"\nREPO: {repo_name}")

        repo = g.get_repo(repo_name)

        try:
            workflow_files = repo.get_contents(".github/workflows")
        except Exception as e:
            print(f" Could not access workflows: {e}")
            continue

```

```

        yml_files = [f for f in workflow_files if f.name.endswith((''.yaml',
        '.yaml'))]

    for wf_file in yml_files:
        content = wf_file.decoded_content.decode('utf-8')

        safe_name = repo_name.replace("/", "_")
        filepath = os.path.join(output_dir, f"{safe_name}_{wf_file.name}")
        with open(filepath, "w") as f:
            f.write(content)

        analysis = analyse_workflow(content, wf_file.name)

        if "error" in analysis:
            print(f"    [{wf_file.name}] Error: {analysis['error']}")
            continue

        print(f"\n    [{wf_file.name}]")
        print(f"        Jobs: {analysis['job_count']}")
        print(f"        Has caching: {'YES' if analysis['has_caching'] else
'NO'})")
        print(f"        Has matrix: {'YES' if analysis['has_matrix'] else
'NO'})")
        print(f"        Has sequential deps: {'YES' if
analysis['has_sequential_deps'] else 'NO'})")

        for job_name, job_info in analysis["jobs"].items():
            needs_str = f" (needs: {job_info['needs']})" if
job_info['needs'] else ""
            matrix_str = " [MATRIX]" if job_info['has_matrix'] else ""
            cache_str = " [CACHED]" if job_info['uses_cache'] else ""
            deps_str = " [INSTALLS DEPS]" if job_info['installs_deps'] else
""
            print(f"        -
{job_name}{needs_str}{matrix_str}{cache_str}{deps_str}")

        if analysis["caching_opportunities"]:
            print(f"        CACHING OPPORTUNITIES:")
            for opp in analysis["caching_opportunities"]:
                print(f"            - {opp}")

        if analysis["parallelization_opportunities"]:
            print(f"        PARALLELIZATION OPPORTUNITIES:")
            for opp in analysis["parallelization_opportunities"]:
                print(f"            - {opp}")

    print(f"\nAll workflow YAML files saved to: {output_dir}/")

if __name__ == "__main__":
    main()

```

Appendix D: Baseline Data Collection Script

This script collects build time data from recent successful workflow runs for each selected repository using the GitHub Actions API.

Filename: stage2_baseline_collection.py

```
import csv
import time
from github import Github, Auth

GITHUB_TOKEN = "YOUR_TOKEN_HERE"

REPOS = {
    "stripe/stripe-python": ["ci.yml"],
    "django/djangoproject.com": ["tests.yml"],
    "pubkey/broadcast-channel": ["main.yml"],
    "nashtech-garage/yas": ["cart-ci.yml", "product-ci.yml", "order-ci.yml",
    "customer-ci.yml", "inventory-ci.yml"],
    "sparrowwallet/sparrow": ["package.yml"],
}

MIN_RUNS = 30

def get_rate_remaining(g):
    try:
        rate = g.get_rate_limit()
        if hasattr(rate, 'core'):
            return rate.core.remaining
        else:
            return rate.rate.remaining
    except Exception:
        return 999

def main():
    auth = Auth.Token(GITHUB_TOKEN)
    g = Github(auth=auth)

    print(f"Authenticated as: {g.get_user().login}")
    print(f"Rate limit remaining: {get_rate_remaining(g)}")

    all_results = []

    for repo_name, workflow_files in REPOS.items():
        print(f"\n{repo_name}")

        repo = g.get_repo(repo_name)

        for wf_name in workflow_files:
            print(f"  Workflow: {wf_name}")

            target_workflow = None
            try:
                for wf in repo.get_workflows():
                    if wf.path.endswith(wf_name):
                        target_workflow = wf
                        break
            except Exception:
                continue

            if not target_workflow:
                continue

            collected = 0
```

```

try:
    runs = target_workflow.get_runs(status="completed")
    for run in runs:
        if collected >= MIN_RUNS:
            break
        if run.conclusion != "success":
            continue

        total_duration = (run.updated_at -
run.created_at).total_seconds()

        jobs_info = []
        try:
            jobs = run.jobs()
            for job in jobs:
                if job.completed_at and job.started_at:
                    duration = (job.completed_at -
job.started_at).total_seconds()
                    jobs_info.append({"name": job.name, "duration":
round(duration, 1)})
        except Exception:
            pass

        jobs_str = "; ".join([f"{j['name']}={j['duration']}s" for j
in jobs_info])

        record = {
            "repository": repo_name,
            "workflow": wf_name,
            "run_number": run.run_number,
            "run_id": run.id,
            "total_duration_seconds": round(total_duration, 1),
            "job_count": len(jobs_info),
            "job_durations": jobs_str,
            "branch": run.head_branch,
            "event": run.event,
            "created_at": run.created_at.strftime("%Y-%m-%d
%H:%M:%S"),
            "conclusion": run.conclusion,
        }

        all_results.append(record)
        collected += 1

        remaining = get_rate_remaining(g)
        if remaining < 50:
            time.sleep(60)

    except Exception as e:
        print(f"    Error: {e}")

    print(f"    Collected: {collected} runs")

output_file = "stage2_baseline_data.csv"
if all_results:
    with open(output_file, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=all_results[0].keys())
        writer.writeheader()
        writer.writerows(all_results)

    print(f"\nTotal records: {len(all_results)}")

```

```

        for repo_name in REPOS:
            repo_records = [r for r in all_results if r["repository"] ==
repo_name]
            if repo_records:
                durations = [r["total_duration_seconds"] for r in repo_records]
                avg = sum(durations) / len(durations)
                sorted_d = sorted(durations)
                median = sorted_d[len(sorted_d)//2]
                variance = sum((d - avg)**2 for d in durations) / len(durations)
                std_dev = variance ** 0.5
                print(f"\n{repo_name}: {len(repo_records)} runs,
mean={avg:.1f}s, median={median:.1f}s, std={std_dev:.1f}s")

            print(f"\nData saved to: {output_file}")

if __name__ == "__main__":
    main()

```

Appendix E: Post-Optimization Data Collection Scripts

These scripts collect build time data from the forked repositories after applying each optimization configuration.

D.1 stripe/stripe-python

Filename: stage5_collect_stripe.py

```

import csv
from github import Github, Auth

GITHUB_TOKEN = "YOUR TOKEN HERE"
FORK_REPO = "SagarGi/stripe-python"

def main():
    auth = Auth.Token(GITHUB_TOKEN)
    g = Github(auth=auth)
    repo = g.get_repo(FORK_REPO)

    all_runs = []
    for run in repo.get_workflow_runs(status="completed"):
        if run.conclusion != "success":
            continue
        duration = (run.updated_at - run.created_at).total_seconds()
        if duration > 3600:
            continue
        all_runs.append({
            "run_id": run.id,
            "run_number": run.run_number,
            "total_duration_seconds": round(duration, 1),
            "created_at": run.created_at.strftime("%Y-%m-%d %H:%M:%S"),
            "commit_message": (run.head_commit.message if run.head_commit else
""[:100],
            "event": run.event,
            "branch": run.head_branch,
        })

```

```

        if len(all_runs) >= 50:
            break

    all_runs.sort(key=lambda x: x["created_at"])

    for i, r in enumerate(all_runs, 1):
        print(f"{i}. Run {r['run_number']}: {r['total_duration_seconds']}s - {r['commit_message'][:50]}")

    output_file = "stage5_stripe_post_optimization.csv"
    if all_runs:
        with open(output_file, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=all_runs[0].keys())
            writer.writeheader()
            writer.writerows(all_runs)
        print(f"\nSaved to: {output_file}")

if __name__ == "__main__":
    main()

```

D.2 django/djangoproject.com

Filename: stage5_collect_django.py

```

import csv
from github import Github, Auth

GITHUB_TOKEN = "YOUR_TOKEN_HERE"
FORK_REPO = "SagarGi/djangoproject.com"

def main():
    auth = Auth.Token(GITHUB_TOKEN)
    g = Github(auth=auth)
    repo = g.get_repo(FORK_REPO)

    all_runs = []
    for run in repo.get_workflow_runs(status="completed"):
        if run.conclusion != "success":
            continue
        duration = (run.updated_at - run.created_at).total_seconds()
        if duration > 3600:
            continue
        all_runs.append({
            "run_id": run.id,
            "run_number": run.run_number,
            "total_duration_seconds": round(duration, 1),
            "created_at": run.created_at.strftime("%Y-%m-%d %H:%M:%S"),
            "commit_message": (run.head_commit.message if run.head_commit else
""[:100],
            "event": run.event,
            "branch": run.head_branch,
        })
        if len(all_runs) >= 30:
            break

    all_runs.sort(key=lambda x: x["created_at"])

    for i, r in enumerate(all_runs, 1):

```

```

        print(f"{i}. Run {r['run_number']}: {r['total_duration_seconds']}s -
        {r['commit_message'][:50]}")

    output_file = "stage5_django_post_optimization.csv"
    if all_runs:
        with open(output_file, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=all_runs[0].keys())
            writer.writeheader()
            writer.writerows(all_runs)
        print(f"\nSaved to: {output_file}")

if __name__ == "__main__":
    main()

```

D.3 nashtech-garage/yas

Filename: stage5_collect_yas.py

```

import csv
from github import Github, Auth

GITHUB_TOKEN = "YOUR_TOKEN_HERE"
FORK_REPO = "SagarGi/yas"

def main():
    auth = Auth.Token(GITHUB_TOKEN)
    g = Github(auth=auth)
    repo = g.get_repo(FORK_REPO)

    all_runs = []
    for run in repo.get_workflow_runs(status="completed"):
        if "cart" not in run.name.lower():
            continue
        duration = (run.updated_at - run.created_at).total_seconds()
        if duration > 3600:
            continue
        all_runs.append({
            "run_id": run.id,
            "run_number": run.run_number,
            "workflow": run.name,
            "total_duration_seconds": round(duration, 1),
            "conclusion": run.conclusion,
            "created_at": run.created_at.strftime("%Y-%m-%d %H:%M:%S"),
            "commit_message": (run.head_commit.message if run.head_commit else
            "")[:100],
            "event": run.event,
            "branch": run.head_branch,
        })
        if len(all_runs) >= 50:
            break

    all_runs.sort(key=lambda x: x["created_at"])

    for i, r in enumerate(all_runs, 1):
        print(f"{i}. Run {r['run_number']}: {r['total_duration_seconds']}s -
        {r['commit_message'][:50]}")

    output_file = "stage5_yas_post_optimization.csv"
    if all_runs:

```

```

        with open(output_file, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=all_runs[0].keys())
            writer.writeheader()
            writer.writerows(all_runs)
        print(f"\nSaved to: {output_file}")

if __name__ == "__main__":
    main()

```

D.4 sparrowwallet/sparrow

Filename: stage5_collect_sparrow.py

```

import csv
from github import Github, Auth

GITHUB_TOKEN = "YOUR TOKEN HERE"
FORK_REPO = "SagarGi/sparrow"

def main():
    auth = Auth.Token(GITHUB_TOKEN)
    g = Github(auth=auth)
    repo = g.get_repo(FORK_REPO)

    all_runs = []
    for run in repo.get_workflow_runs(status="completed"):
        if run.conclusion != "success":
            continue
        duration = (run.updated_at - run.created_at).total_seconds()
        if duration > 3600:
            continue
        all_runs.append({
            "run_id": run.id,
            "run_number": run.run_number,
            "total_duration_seconds": round(duration, 1),
            "conclusion": run.conclusion,
            "created_at": run.created_at.strftime("%Y-%m-%d %H:%M:%S"),
            "commit_message": (run.head_commit.message if run.head_commit else
""[:100],
            "event": run.event,
            "branch": run.head_branch,
        })
        if len(all_runs) >= 30:
            break

    all_runs.sort(key=lambda x: x["created_at"])

    for i, r in enumerate(all_runs, 1):
        print(f"{i}. Run {r['run_number']}: {r['total_duration_seconds']}s -
{r['commit_message'][:50]}")

    output_file = "stage5_sparrow_post_optimization.csv"
    if all_runs:
        with open(output_file, "w", newline="", encoding="utf-8") as f:
            writer = csv.DictWriter(f, fieldnames=all_runs[0].keys())
            writer.writeheader()
            writer.writerows(all_runs)
        print(f"\nSaved to: {output_file}")

```

```
if __name__ == "__main__":
    main()
```

Appendix E: Optimized Workflow Configurations

The following are the modified GitHub Actions workflow files used to test each optimization strategy on the forked repositories.

E.1 stripe/stripe-python — Caching

Filename: `.github/workflows/ci.yml`

```
name: CI
on:
  workflow_dispatch: {}
  push:
    branches:
      - master
      - beta
      - sdk-release/**
      - feature/**
    tags:
      - v[0-9]+.[0-9]+.[0-9]+*
  pull_request:
    branches:
      - master
      - beta
      - private-preview
      - sdk-release/**
      - feature/**
permissions: {}
jobs:
  lint:
    name: Static Checks
    runs-on: "ubuntu-24.04"
    permissions:
      contents: read
    steps:
      - uses: extractions/setup-just@v2
      - uses: actions/checkout@v3
      - name: Set up Python 3
        uses: actions/setup-python@v4
        with:
          python-version: "3.10"
      - name: Cache pip dependencies
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${{ runner.os }}-pip-lint-${{ hashFiles('**/setup.py', '**/pyproject.toml') }}
          restore-keys: |
            ${{ runner.os }}-pip-lint-
            ${{ runner.os }}-pip-
      - name: check examples w/ mypy (against python@3.10)
        run: just typecheck-examples
      - name: check linting
        run: just --no-deps lint
      - name: check formatting
```

```

    run: just --no-deps format-check
  - name: check types (all Python versions)
    run: |
      set -eox
      for minor in {6..12}; do
        just --no-deps typecheck $minor
      done
build:
  name: Build
  runs-on: "ubuntu-24.04"
  permissions:
    contents: read
  steps:
    - uses: extractions/setup-just@v2
    - uses: actions/checkout@v3
    - name: Set up Python 3
      uses: actions/setup-python@v4
      with:
        python-version: "3.10"
    - name: Cache pip dependencies
      uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: ${{ runner.os }}-pip-build-${{ hashFiles('**/setup.py',
          '**/pyproject.toml') }}
        restore-keys: |
          ${{ runner.os }}-pip-build-
          ${{ runner.os }}-pip-
    - name: Build and check package
      run: |
        just build
    - name: "Upload Artifact"
      uses: actions/upload-artifact@v4
      with:
        name: dist
        path: dist/
test:
  runs-on: ubuntu-22.04
  permissions:
    contents: read
  strategy:
    fail-fast: false
  matrix:
    python_version:
      - "3.9"
      - "3.10"
      - "3.11"
      - "3.12"
      - "3.13"
      - "3.14"
      - "pypy-3.9"
      - "pypy-3.10"
      - "pypy-3.11"
  name: Test (${{ matrix.python_version }})
  steps:
    - uses: extractions/setup-just@v2
    - uses: actions/checkout@v3
    - name: Set up Python ${{ matrix.python_version }}
      uses: actions/setup-python@v4
      with:
        python-version: ${{ matrix.python_version }}

```

```

- name: Cache pip dependencies
  uses: actions/cache@v4
  with:
    path: ~/.cache/pip
    key: ${{ runner.os }}-pip-test-${{ matrix.python_version }}-${{
hashFiles('**/setup.py', '**/pyproject.toml') }}
    restore-keys: |
      ${{ runner.os }}-pip-test-${{ matrix.python_version }}-
      ${{ runner.os }}-pip-
- uses: stripe/openapi/actions/stripe-mock@master
- name: "run tests"
  run: just test
publish:
  name: Publish
  if: >-
    ((github.event_name == 'workflow_dispatch') || (github.event_name ==
'push')) &&
    startsWith(github.ref, 'refs/tags/v')
  needs: [build, test, lint]
  runs-on: "ubuntu-24.04"
  permissions:
    contents: read
  steps:
    - uses: extractions/setup-just@v2
    - uses: actions/checkout@v3
    - name: Download all workflow run artifacts
      uses: actions/download-artifact@v4
      with:
        name: dist
        path: dist
    - name: Set up Python 3
      uses: actions/setup-python@v4
      with:
        python-version: "3.10"
    - name: Cache pip dependencies
      uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: ${{ runner.os }}-pip-publish-${{ hashFiles('**/setup.py',
'**/pyproject.toml') }}
        restore-keys: |
          ${{ runner.os }}-pip-publish-
          ${{ runner.os }}-pip-
    - name: Install tools
      run: just install-build-deps
    - name: Publish packages to PyPI
      run: |
        set -ex
        source .venv/bin/activate
        python -m twine upload --verbose dist/*
      env:
        TWINE_USERNAME: ${ secrets.TWINE_USERNAME }
        TWINE_PASSWORD: ${ secrets.TWINE_PASSWORD }
    - uses: stripe/openapi/actions/notify-release@master
      if: always()
      with:
        bot_token: ${ secrets.SLACK_BOT_TOKEN }

```

E.2 stripe/stripe-python — Parallelization

Filename: *.github/workflows/ci.yml*

```

name: CI
on:
  workflow_dispatch: {}
  push:
    branches:
      - master
      - beta
      - sdk-release/**
      - feature/**
    tags:
      - v[0-9]+.[0-9]+.[0-9]+*
  pull_request:
    branches:
      - master
      - beta
      - private-preview
      - sdk-release/**
      - feature/**
permissions: {}
jobs:
  lint:
    name: Static Checks
    runs-on: "ubuntu-24.04"
    permissions:
      contents: read
    steps:
      - uses: extractions/setup-just@v2
      - uses: actions/checkout@v3
      - name: Set up Python 3
        uses: actions/setup-python@v4
        with:
          python-version: "3.10"
      - name: check examples w/ mypy (against python@3.10)
        run: just typecheck-examples
      - name: check linting
        run: just --no-deps lint
      - name: check formatting
        run: just --no-deps format-check
      - name: check types (all Python versions)
        run: |
          set -eox
          for minor in {6..12}; do
            just --no-deps typecheck $minor
          done
  build:
    name: Build
    runs-on: "ubuntu-24.04"
    permissions:
      contents: read
    steps:
      - uses: extractions/setup-just@v2
      - uses: actions/checkout@v3
      - name: Set up Python 3
        uses: actions/setup-python@v4
        with:
          python-version: "3.10"
      - name: Build and check package
        run: |
          just build
      - name: "Upload Artifact"
        uses: actions/upload-artifact@v4

```

```

    with:
      name: dist
      path: dist/
test:
  runs-on: ubuntu-22.04
  permissions:
    contents: read
  strategy:
    fail-fast: false
  matrix:
    python_version:
      - "3.9"
      - "3.10"
      - "3.11"
      - "3.12"
      - "3.13"
      - "3.14"
      - "pypy-3.9"
      - "pypy-3.10"
      - "pypy-3.11"
  name: Test (${{ matrix.python_version }})
  steps:
    - uses: extractions/setup-just@v2
    - uses: actions/checkout@v3
    - name: Set up Python ${{ matrix.python_version }}
      uses: actions/setup-python@v4
      with:
        python-version: ${{ matrix.python_version }}
    - uses: stripe/openapi/actions/stripe-mock@master
    - name: "run tests"
      run: just test
typecheck:
  name: Type Check
  runs-on: "ubuntu-24.04"
  permissions:
    contents: read
  strategy:
    fail-fast: false
  matrix:
    python_minor: ["6", "7", "8", "9", "10", "11", "12"]
  steps:
    - uses: extractions/setup-just@v2
    - uses: actions/checkout@v3
    - name: Set up Python 3
      uses: actions/setup-python@v4
      with:
        python-version: "3.10"
    - name: Install dependencies
      run: just typecheck-examples
    - name: Type check Python 3.${{ matrix.python_minor }}
      run: just --no-deps typecheck ${{ matrix.python_minor }}
publish:
  name: Publish
  if: >-
    ((github.event_name == 'workflow_dispatch') || (github.event_name ==
'push')) &&
    startsWith(github.ref, 'refs/tags/v')
  needs: [build, test, lint, typecheck]
  runs-on: "ubuntu-24.04"
  permissions:
    contents: read

```

```

steps:
  - uses: extractions/setup-just@v2
  - uses: actions/checkout@v3
  - name: Download all workflow run artifacts
    uses: actions/download-artifact@v4
    with:
      name: dist
      path: dist
  - name: Set up Python 3
    uses: actions/setup-python@v4
    with:
      python-version: "3.10"
  - name: Install tools
    run: just install-build-deps
  - name: Publish packages to PyPI
    run: |
      set -ex
      source .venv/bin/activate
      python -m twine upload --verbose dist/*
    env:
      TWINE_USERNAME: ${ secrets.TWINE_USERNAME }
      TWINE_PASSWORD: ${ secrets.TWINE_PASSWORD }
  - uses: stripe/openapi/actions/notify-release@master
    if: always()
    with:
      bot_token: ${ secrets.SLACK_BOT_TOKEN }

```

E.3 stripe/stripe-python — Combined

Filename: .github/workflows/ci.yml

```

name: CI
on:
  workflow_dispatch: {}
  push:
    branches:
      - master
      - beta
      - sdk-release/**
      - feature/**
    tags:
      - v[0-9]+.[0-9]+.[0-9]+*
  pull_request:
    branches:
      - master
      - beta
      - private-preview
      - sdk-release/**
      - feature/**
permissions: {}
jobs:

```

```

lint:
  name: Static Checks
  runs-on: "ubuntu-24.04"
  permissions:
    contents: read
  steps:
    - uses: extractions/setup-just@v2
    - uses: actions/checkout@v3
    - name: Set up Python 3
      uses: actions/setup-python@v4
      with:
        python-version: "3.10"
    - name: Cache pip dependencies
      uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: ${ runner.os }}-pip-lint-${ hashFiles('**/setup.py',
'**/pyproject.toml') }}
        restore-keys: |
          ${ runner.os }}-pip-lint-
          ${ runner.os }}-pip-
    - name: check examples w/ mypy (against python@3.10)
      run: just typecheck-examples
    - name: check linting
      run: just --no-deps lint
    - name: check formatting
      run: just --no-deps format-check
build:
  name: Build
  runs-on: "ubuntu-24.04"
  permissions:
    contents: read
  steps:
    - uses: extractions/setup-just@v2
    - uses: actions/checkout@v3
    - name: Set up Python 3
      uses: actions/setup-python@v4
      with:
        python-version: "3.10"
    - name: Cache pip dependencies
      uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: ${ runner.os }}-pip-build-${ hashFiles('**/setup.py',
'**/pyproject.toml') }}
        restore-keys: |
          ${ runner.os }}-pip-build-
          ${ runner.os }}-pip-
    - name: Build and check package
      run: |
        just build
    - name: "Upload Artifact"
      uses: actions/upload-artifact@v4
      with:
        name: dist
        path: dist/
test:
  runs-on: ubuntu-22.04
  permissions:
    contents: read
  strategy:

```

```

fail-fast: false
matrix:
  python_version:
    - "3.9"
    - "3.10"
    - "3.11"
    - "3.12"
    - "3.13"
    - "3.14"
    - "pypy-3.9"
    - "pypy-3.10"
    - "pypy-3.11"
name: Test (${{ matrix.python_version }})
steps:
  - uses: extractions/setup-just@v2
  - uses: actions/checkout@v3
  - name: Set up Python ${{ matrix.python_version }}
    uses: actions/setup-python@v4
    with:
      python-version: ${{ matrix.python_version }}
  - name: Cache pip dependencies
    uses: actions/cache@v4
    with:
      path: ~/.cache/pip
      key: ${{ runner.os }}-pip-test-${{ matrix.python_version }}-${{
hashFiles('**/setup.py', '**/pyproject.toml') }}
      restore-keys: |
        ${{ runner.os }}-pip-test-${{ matrix.python_version }}-
        ${{ runner.os }}-pip-
  - uses: stripe/openapi/actions/stripe-mock@master
  - name: "run tests"
    run: just test
typecheck:
  name: Type Check
  runs-on: "ubuntu-24.04"
  permissions:
    contents: read
  strategy:
    fail-fast: false
    matrix:
      python_minor: ["6", "7", "8", "9", "10", "11", "12"]
  steps:
    - uses: extractions/setup-just@v2
    - uses: actions/checkout@v3
    - name: Set up Python 3
      uses: actions/setup-python@v4
      with:
        python-version: "3.10"
    - name: Cache pip dependencies
      uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: ${{ runner.os }}-pip-typecheck-${{ hashFiles('**/setup.py',
'**/pyproject.toml') }}
        restore-keys: |
          ${{ runner.os }}-pip-typecheck-
          ${{ runner.os }}-pip-
    - name: Install dependencies
      run: just typecheck-examples
    - name: Type check Python 3.${{ matrix.python_minor }}
      run: just --no-deps typecheck ${{ matrix.python_minor }}

```

```

publish:
  name: Publish
  if: >-
    ((github.event_name == 'workflow_dispatch') || (github.event_name ==
'push')) &&
    startsWith(github.ref, 'refs/tags/v')
  needs: [build, test, lint, typecheck]
  runs-on: "ubuntu-24.04"
  permissions:
    contents: read
  steps:
    - uses: extractions/setup-just@v2
    - uses: actions/checkout@v3
    - name: Download all workflow run artifacts
      uses: actions/download-artifact@v4
      with:
        name: dist
        path: dist
    - name: Set up Python 3
      uses: actions/setup-python@v4
      with:
        python-version: "3.10"
    - name: Cache pip dependencies
      uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: ${{ runner.os }}-pip-publish-${{ hashFiles('**/setup.py',
'**/pyproject.toml') }}
        restore-keys: |
          ${{ runner.os }}-pip-publish-
          ${{ runner.os }}-pip-
    - name: Install tools
      run: just install-build-deps
    - name: Publish packages to PyPI
      run: |
        set -ex
        source .venv/bin/activate
        python -m twine upload --verbose dist/*
      env:
        TWINE_USERNAME: ${{ secrets.TWINE_USERNAME }}
        TWINE_PASSWORD: ${{ secrets.TWINE_PASSWORD }}
    - uses: stripe/openapi/actions/notify-release@master
      if: always()
      with:
        bot_token: ${{ secrets.SLACK_BOT_TOKEN }}

```

E.4 django/djangoproject.com — Caching

Filename: .github/workflows/tests.yml

```

---
name: Tests
on:
  push:
    branches:
      - main
  pull_request:
env:
  PYTHON_VERSION: 3.12
jobs:
  tests:

```

```

runs-on: ubuntu-24.04
services:
  postgres:
    image: postgres:17-trixie
    env:
      POSTGRES_PASSWORD: postgres
    options: >-
      --health-cmd pg_isready
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5
    ports:
      - 5432:5432
steps:
  - uses: actions/checkout@v6
  - name: Set up Python
    uses: actions/setup-python@v6
    with:
      python-version: ${ env.PYTHON_VERSION }
  - name: Cache pip dependencies
    uses: actions/cache@v4
    with:
      path: ~/.cache/pip
      key: ${ runner.os }}-pip-${
hashFiles('requirements/tests.txt') }}
      restore-keys: |
        ${ runner.os }}-pip-
  - name: Cache Playwright browsers
    uses: actions/cache@v4
    with:
      path: ~/.cache/ms-playwright
      key: ${ runner.os }}-playwright-${
hashFiles('requirements/tests.txt') }}
      restore-keys: |
        ${ runner.os }}-playwright-
  - name: Install gettext
    run: sudo apt-get update && sudo apt-get install -y gettext
  - name: Install dependencies
    run: |
      python -m pip install --upgrade pip setuptools
      python -m pip install -r requirements/tests.txt
  - name: Install playwright browsers
    run: |
      python -m playwright install --with-deps chromium
  - name: Set up databases
    run: |
      PGPASSWORD="postgres" createuser -U postgres -d django
--superuser -h localhost
      PGPASSWORD="postgres" createdb -U postgres -O django
django -h localhost
      PGPASSWORD="postgres" createuser -U postgres -d
code.django --superuser -h localhost
      PGPASSWORD="postgres" createdb -U postgres -O
code.django code.django -h localhost
      PGPASSWORD="postgres" psql -U postgres -h localhost -c "ALTER
USER django WITH PASSWORD 'secret';"
      PGPASSWORD="postgres" psql -U postgres -h localhost -c "ALTER
USER \"code.django\" WITH PASSWORD 'secret';"
  - name: Load Trac data
    run: |

```

```

PGPASSWORD="postgres" psql -U postgres -d code.djangoproject
-h localhost < tracdb/trac.sql
- name: Create secrets.json
  working-directory: ..
  run: |
    mkdir conf
    echo '{"db_host": "localhost", ' > conf/secrets.json
    echo '"db_password": "secret", ' >> conf/secrets.json
    echo '"trac_db_host": "localhost", ' >> conf/secrets.json
    echo '"trac_db_password": "secret", ' >> conf/secrets.json
    echo '"secret_key": "a"}' >> conf/secrets.json
- name: Run tests
  run: |
    make ci
    coverage xml
- name: Upload coverage reports to Codecov
  uses: codecov/codecov-action@v6
  with:
    token: ${ secrets.CODECOV_TOKEN }

```

E.5 nashtech-garage/yas — Caching (cart service)

Filename: .github/workflows/cart-ci.yaml

```

name: cart service ci
on:
  push:
    branches: [ "main" ]
    paths:
      - "cart/**"
      - ".github/workflows/actions/action.yaml"
      - ".github/workflows/cart-ci.yaml"
      - "pom.xml"
  pull_request:
    branches: [ "main" ]
    paths:
      - "cart/**"
      - ".github/workflows/actions/action.yaml"
      - ".github/workflows/cart-ci.yaml"
      - "pom.xml"
  workflow_dispatch:
jobs:
  Build:
    runs-on: ubuntu-latest
    env:
      FROM_ORIGINAL_REPOSITORY: ${
github.event.pull_request.head.repo.full_name == github.repository || github.ref
== 'refs/heads/main' }
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: ./github/workflows/actions
      - name: Cache Maven dependencies
        uses: actions/cache@v4
        with:
          path: ~/.m2/repository
          key: ${ runner.os }-maven-cart-${ hashFiles('**/pom.xml') }
          restore-keys: |
            ${ runner.os }-maven-cart-
            ${ runner.os }-maven-

```

```

- name: Run Maven Build Command
  run: mvn clean install -pl cart -am
- name: Run Maven Checkstyle
  if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
  run: mvn checkstyle:checkstyle -pl cart -am
-Dcheckstyle.output.file=cart-checkstyle-result.xml
- name: Upload Checkstyle Result
  if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
  uses: jwgmeligmeyling/checkstyle-github-action@master
  with:
    path: '**/cart-checkstyle-result.xml'
- name: Test Results
  uses: dorny/test-reporter@v1
  if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' } && (success() ||
failure()) }}
  with:
    name: Cart-Service-Unit-Test-Results
    path: "cart/**/*-reports/TEST*.xml"
    reporter: java-junit
- name: OWASP Dependency Check
  if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
  uses: dependency-check/Dependency-Check_Action@main
  env:
    JAVA_HOME: /opt/jdk
  with:
    project: 'yas'
    path: '.'
    format: 'HTML'
- name: Upload OWASP Dependency Check results
  if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
  uses: actions/upload-artifact@master
  with:
    name: OWASP Dependency Check Report
    path: ${ github.workspace }/reports
- name: Analyze with sonar cloud
  if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
  env:
    SONAR_TOKEN: ${ secrets.SONAR_TOKEN }}
  run: mvn org.sonarsource.scanner.maven:sonar-maven-plugin:sonar -pl cart
-am
- name: Add coverage report to PR
  uses: madrapps/jacoco-report@v1.6.1
  if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
  with:
    paths: ${ github.workspace }/cart/target/site/jacoco/jacoco.xml
    token: ${ secrets.GITHUB_TOKEN }}
    min-coverage-overall: 80
    min-coverage-changed-files: 60
    title: 'Cart Coverage Report'
    update-comment: true
- name: Log in to the Container registry
  if: ${ github.ref == 'refs/heads/main' }}
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${ github.actor }}
    password: ${ secrets.GITHUB_TOKEN }}
- name: Build and push Docker images
  if: ${ github.ref == 'refs/heads/main' }}
  uses: docker/build-push-action@v6
  with:

```

```

context: ./cart
push: true
tags: ghcr.io/nashtech-garage/yas-cart:latest

```

E.6 nashtech-garage/yas — Parallelization (cart service)

Filename: .github/workflows/cart-ci.yaml

```

name: cart service ci
on:
  push:
    branches: [ "main" ]
    paths:
      - "cart/**"
      - ".github/workflows/actions/action.yaml"
      - ".github/workflows/cart-ci.yaml"
      - "pom.xml"
  pull_request:
    branches: [ "main" ]
    paths:
      - "cart/**"
      - ".github/workflows/actions/action.yaml"
      - ".github/workflows/cart-ci.yaml"
      - "pom.xml"
  workflow_dispatch:
jobs:
  build-and-test:
    name: Build & Test
    runs-on: ubuntu-latest
    env:
      FROM_ORIGINAL_REPOSITORY: ${
github.event.pull_request.head.repo.full_name == github.repository || github.ref
== 'refs/heads/main' }}
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: ./github/workflows/actions
      - name: Run Maven Build Command
        run: mvn clean install -pl cart -am
      - name: Test Results
        uses: dorny/test-reporter@v1
        if: ${
env.FROM_ORIGINAL_REPOSITORY == 'true' && (success() ||
failure()) }
        with:
          name: Cart-Service-Unit-Test-Results
          path: "cart/**/*-reports/TEST*.xml"
          reporter: java-junit
      - name: Upload build artifacts
        uses: actions/upload-artifact@v4
        with:
          name: cart-build
          path: cart/target/
          retention-days: 1
    checkstyle:
      name: Checkstyle Analysis
      runs-on: ubuntu-latest
      env:
        FROM_ORIGINAL_REPOSITORY: ${
github.event.pull_request.head.repo.full_name == github.repository || github.ref
== 'refs/heads/main' }}

```

```

steps:
  - uses: actions/checkout@v4
    with:
      fetch-depth: 0
  - uses: ../github/workflows/actions
  - name: Run Maven Checkstyle
    if: ${{ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
    run: mvn checkstyle:checkstyle -pl cart -am
-Dcheckstyle.output.file=cart-checkstyle-result.xml
  - name: Upload Checkstyle Result
    if: ${{ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
    uses: jwgmeligmeyling/checkstyle-github-action@master
    with:
      path: '**/cart-checkstyle-result.xml'
security-scan:
  name: Security & Quality
  runs-on: ubuntu-latest
  env:
    FROM_ORIGINAL_REPOSITORY: ${{
github.event.pull_request.head.repo.full_name == github.repository || github.ref
== 'refs/heads/main' }}
  steps:
    - uses: actions/checkout@v4
      with:
        fetch-depth: 0
    - uses: ../github/workflows/actions
    - name: Run Maven Build
      run: mvn clean install -pl cart -am -DskipTests
    - name: Analyze with sonar cloud
      continue-on-error: true
      if: ${{ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
      env:
        SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
      run: mvn org.sonarsource.scanner.maven:sonar-maven-plugin:sonar -pl cart
-am
  - name: OWASP Dependency Check
    if: ${{ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
    uses: dependency-check/Dependency-Check_Action@main
    env:
      JAVA_HOME: /opt/jdk
    with:
      project: 'yas'
      path: '.'
      format: 'HTML'
  - name: Upload OWASP Dependency Check results
    if: ${{ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
    uses: actions/upload-artifact@master
    with:
      name: OWASP Dependency Check Report
      path: ${{github.workspace}}/reports
coverage:
  name: Coverage Report
  needs: [build-and-test]
  runs-on: ubuntu-latest
  if: ${{ github.event.pull_request.head.repo.full_name == github.repository
|| github.ref == 'refs/heads/main' }}
  steps:
    - uses: actions/checkout@v4
    - name: Download build artifacts
      uses: actions/download-artifact@v4
      with:

```

```

      name: cart-build
      path: cart/target/
- name: Add coverage report to PR
  uses: madrapps/jacoco-report@v1.6.1
  with:
    paths: ${GITHUB_WORKSPACE}/cart/target/site/jacoco/jacoco.xml
    token: ${GITHUB_TOKEN}
    min-coverage-overall: 80
    min-coverage-changed-files: 60
    title: 'Cart Coverage Report'
    update-comment: true
docker:
  name: Docker Build & Push
  needs: [build-and-test]
  runs-on: ubuntu-latest
  if: GITHUB_REF == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4
    - name: Download build artifacts
      uses: actions/download-artifact@v4
      with:
        name: cart-build
        path: cart/target/
    - name: Log in to the Container registry
      continue-on-error: true
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: GITHUB_ACTOR
        password: GITHUB_TOKEN
    - name: Build and push Docker images
      continue-on-error: true
      uses: docker/build-push-action@v6
      with:
        context: ./cart
        push: true
        tags: ghcr.io/nashtech-garage/yas-cart:latest

```

E.7 nashtech-garage/yas — Combined (cart service)

Filename: .github/workflows/cart-ci.yaml

```

name: cart service ci
on:
  push:
    branches: [ "main" ]
    paths:
      - "cart/**"
      - ".github/workflows/actions/action.yaml"
      - ".github/workflows/cart-ci.yaml"
      - "pom.xml"
  pull_request:
    branches: [ "main" ]
    paths:
      - "cart/**"
      - ".github/workflows/actions/action.yaml"
      - ".github/workflows/cart-ci.yaml"
      - "pom.xml"
workflow_dispatch:

```

```

jobs:
  build-and-test:
    name: Build & Test
    runs-on: ubuntu-latest
    env:
      FROM_ORIGINAL_REPOSITORY: ${
github.event.pull_request.head.repo.full_name == github.repository || github.ref
== 'refs/heads/main' }}
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: ./github/workflows/actions
      - name: Cache Maven dependencies
        uses: actions/cache@v4
        with:
          path: ~/.m2/repository
          key: ${ runner.os }}-maven-cart-${ hashFiles('**/pom.xml') }}
          restore-keys: |
            ${ runner.os }}-maven-cart-
            ${ runner.os }}-maven-
      - name: Run Maven Build Command
        run: mvn clean install -pl cart -am
      - name: Test Results
        uses: dorny/test-reporter@v1
        if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' && (success() ||
failure()) }}
        with:
          name: Cart-Service-Unit-Test-Results
          path: "cart/**/*-reports/TEST*.xml"
          reporter: java-junit
      - name: Upload build artifacts
        uses: actions/upload-artifact@v4
        with:
          name: cart-build
          path: cart/target/
          retention-days: 1
    checkstyle:
      name: Checkstyle Analysis
      runs-on: ubuntu-latest
      env:
        FROM_ORIGINAL_REPOSITORY: ${
github.event.pull_request.head.repo.full_name == github.repository || github.ref
== 'refs/heads/main' }}
      steps:
        - uses: actions/checkout@v4
          with:
            fetch-depth: 0
        - uses: ./github/workflows/actions
        - name: Cache Maven dependencies
          uses: actions/cache@v4
          with:
            path: ~/.m2/repository
            key: ${ runner.os }}-maven-cart-checkstyle-${
hashFiles('**/pom.xml') }}
            restore-keys: |
              ${ runner.os }}-maven-cart-
              ${ runner.os }}-maven-
        - name: Run Maven Checkstyle
          if: ${ env.FROM_ORIGINAL_REPOSITORY == 'true' }}

```

```

        run: mvn checkstyle:checkstyle -pl cart -am
-Dcheckstyle.output.file=cart-checkstyle-result.xml
    - name: Upload Checkstyle Result
      if: ${{ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
      uses: jwgmeligmeyling/checkstyle-github-action@master
      with:
        path: '**/cart-checkstyle-result.xml'
  security-scan:
    name: Security & Quality
    runs-on: ubuntu-latest
    env:
      FROM_ORIGINAL_REPOSITORY: ${{
github.event.pull_request.head.repo.full_name == github.repository || github.ref
== 'refs/heads/main' }}
    steps:
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0
      - uses: ../github/workflows/actions
      - name: Cache Maven dependencies
        uses: actions/cache@v4
        with:
          path: ~/.m2/repository
          key: ${{ runner.os }}-maven-cart-security-${{ hashFiles '**/pom.xml' }}
    }}

    restore-keys: |
      ${{ runner.os }}-maven-cart-
      ${{ runner.os }}-maven-
  - name: Run Maven Build
    run: mvn clean install -pl cart -am -DskipTests
  - name: Analyze with sonar cloud
    continue-on-error: true
    if: ${{ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
    env:
      SONAR_TOKEN: ${{ secrets.SONAR_TOKEN }}
    run: mvn org.sonarsource.scanner.maven:sonar-maven-plugin:sonar -pl cart
-am
  - name: OWASP Dependency Check
    if: ${{ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
    uses: dependency-check/Dependency-Check_Action@main
    env:
      JAVA_HOME: /opt/jdk
    with:
      project: 'yas'
      path: '.'
      format: 'HTML'
  - name: Upload OWASP Dependency Check results
    if: ${{ env.FROM_ORIGINAL_REPOSITORY == 'true' }}
    uses: actions/upload-artifact@master
    with:
      name: OWASP Dependency Check Report
      path: ${{github.workspace}}/reports
  coverage:
    name: Coverage Report
    needs: [build-and-test]
    runs-on: ubuntu-latest
    if: ${{ github.event.pull_request.head.repo.full_name == github.repository
|| github.ref == 'refs/heads/main' }}
    steps:
      - uses: actions/checkout@v4
      - name: Download build artifacts

```

```

    uses: actions/download-artifact@v4
    with:
      name: cart-build
      path: cart/target/
  - name: Add coverage report to PR
    uses: madrapps/jacoco-report@v1.6.1
    with:
      paths: ${GITHUB_WORKSPACE}/cart/target/site/jacoco/jacoco.xml
      token: ${GITHUB_TOKEN}
      min-coverage-overall: 80
      min-coverage-changed-files: 60
      title: 'Cart Coverage Report'
      update-comment: true
docker:
  name: Docker Build & Push
  needs: [build-and-test]
  runs-on: ubuntu-latest
  if: GITHUB_REF == 'refs/heads/main'
  steps:
    - uses: actions/checkout@v4
    - name: Download build artifacts
      uses: actions/download-artifact@v4
      with:
        name: cart-build
        path: cart/target/
    - name: Log in to the Container registry
      continue-on-error: true
      uses: docker/login-action@v3
      with:
        registry: ghcr.io
        username: GITHUB_ACTOR
        password: GITHUB_TOKEN
    - name: Build and push Docker images
      continue-on-error: true
      uses: docker/build-push-action@v6
      with:
        context: ./cart
        push: true
        tags: ghcr.io/nashtech-garage/vas-cart:latest

```

E.8 sparrowwallet/sparrow — Caching

Filename: *.github/workflows/package.yaml*

```

name: Package
on: workflow_dispatch
permissions:
  contents: read
jobs:
  build:
    runs-on: ${{ matrix.os }}
    strategy:
      matrix:
        os: [windows-2022, ubuntu-22.04, ubuntu-22.04-arm, macos-15-intel,
macos-14]
    steps:
      - uses: actions/checkout@v6
        with:
          submodules: recursive
      - name: Clear Java tool-cache for reproducibility
        shell: bash
        run: rm -rf "$RUNNER_TOOL_CACHE"/Java_*
      - name: Set up JDK 25.0.2

```

```

    uses: actions/setup-java@v5
    with:
      distribution: 'temurin'
      java-version: '25.0.2'
  - name: Cache Gradle dependencies
    uses: actions/cache@v4
    with:
      path: |
        ~/.gradle/caches
        ~/.gradle/wrapper
      key: ${runner.os}-${runner.arch}-gradle-${hashFiles('**/*.gradle*', '**/gradle-wrapper.properties')}
      restore-keys: |
        ${runner.os}-${runner.arch}-gradle-
  - name: Show Build Versions
    run: ./gradlew -v
  - name: Build with Gradle
    run: ./gradlew jpackage
  - name: Codesign, package and notarize macOS distribution
    if: ${runner.os == 'macOS'}
    uses: sparrowwallet/github-actions/codesign-macos@v1
    with:
      app-name: Sparrow
      certificate: ${secrets.MACOS_CERTIFICATE}
      certificate-password: ${secrets.MACOS_CERTIFICATE_PASSWORD}
      apple-id: ${secrets.MACOS_NOTARIZATION_APPLE_ID}
      team-id: ${secrets.MACOS_NOTARIZATION_TEAM_ID}
      notarization-password: ${secrets.MACOS_NOTARIZATION_PASSWORD}
  - name: Package Windows zip distribution
    if: ${runner.os == 'Windows'}
    run: ./gradlew packageZipDistribution
  - name: Package Linux tar distribution
    if: ${runner.os == 'Linux'}
    run: ./gradlew packageTarDistribution
  - name: Repackage Linux deb distribution
    if: ${runner.os == 'Linux'}
    run: ./repackage.sh
  - name: Upload Artifact
    uses: actions/upload-artifact@v6
    with:
      name: Sparrow Build - ${runner.os} ${runner.arch}
      path: |
        build/jpackage/*
        !build/jpackage/Sparrow/
        !build/jpackage/Sparrow.app/
  - name: Headless build with Gradle
    if: ${runner.os == 'Linux'}
    run: ./gradlew -Djava.awt.headless=true clean jpackage
  - name: Package Linux headless tar distribution
    if: ${runner.os == 'Linux'}
    run: ./gradlew -Djava.awt.headless=true packageTarDistribution
  - name: Repackage Linux headless deb distribution
    if: ${runner.os == 'Linux'}
    run: ./repackage.sh
  - name: Upload Headless Artifact
    if: ${runner.os == 'Linux'}
    uses: actions/upload-artifact@v6
    with:
      name: Sparrow Build - ${runner.os} ${runner.arch} Headless
      path: |
        build/jpackage/*

```

!build/jpackage/Sparrow/